Benchmarking Rule-Based, Classical Machine Learning, and Deep Learning Anomaly Detectors in ETL Data Quality Pipelines

Theo Bigand
ESILV — M2 Data & Artificial Intelligence
Léonard de Vinci Pôle Universitaire
Courbevoie, France
theobigand@gmail.com

Abstract*—Modern data pipelines increasingly depend on automated data quality controls, yet Extract–Transform–Load (ETL) practitioners still rely mostly on hand-crafted declarative rules. Recent work has started to introduce machine learning (ML) anomaly detectors inside ETL workflows, but systematic, apples-to-apples comparisons between rule-based, classical ML, and deep learning approaches remain scarce—especially when evaluated within a realistic pipeline and broken down by the type of anomaly being injected. This paper presents an open-source ETL pipeline in which anomaly detectors from different families share one `fit`/`predict`/`score` interface and are scored on the same pipeline path: three rule-based detectors (Z-score, IQR, Great Expectations), eight classical ML detectors (Isolation Forest, LOF, One-Class SVM, ECOD, COPOD, HBOS, KNN, PCA) and six neural ones (autoencoder, VAE, Deep SVDD, DIF, AE1SVM, LUNAR), with an LSTM autoencoder studied apart. I evaluate the detectors, over five seeds on a stratified held-out split, on real open datasets and on controlled synthetic ones. The real data is a pool of 152 open datasets, used as 100 labelled problems and as 42 tables into which I inject anomalies; three of them are worked out as examples because they show three regimes: Kaggle Credit Card Fraud, a rare-anomaly Cardio dataset and a common-class Water Quality dataset. The synthetic data are generated with Faker, with six distinct anomaly types injected at controlled rates. Beyond aggregate precision, recall, F1, ROC-AUC and PR-AUC, I report a per-anomaly-type recall and precision, precision-recall curves, and a sweep of the injection severity. The aggregate metrics make the families look interchangeable; the per-type view shows each has a narrow, largely non-overlapping competence, so broad coverage needs several families rather than the single best one. A larger registry does not change this: on the 100 labelled problems no detector is clearly ahead, the column-wise ECOD, COPOD and HBOS match Isolation Forest, and the neural models do not beat the best classical one. My goal is a reproducible, pipeline-integrated benchmark that answers the practical question: *which family of detectors should I use, for which kind of data quality problem, at what operational cost?

***Index Terms*—data quality, ETL, anomaly detection, benchmarking, machine learning, Great Expectations, Isolation Forest, autoencoder**

I. Introduction

Extract–Transform–Load (ETL) pipelines are the backbone of modern data infrastructure: they move raw operational data into analytical stores where it feeds dashboards, reports, and machine-learning applications. Organizations increasingly manage data as an asset that must be governed [1], and the value delivered downstream is bounded by the quality of the data that reaches the destination. This is why data quality has long been a first-class concern for data engineers, and why automated, continuous quality control is a capability rather than an option.

The dominant approach in industry remains rule-based: practitioners hand-write declarative expectations—"column `amount` must be positive", "column `email` must match a regex"—in frameworks such as Great Expectations, Deequ, or Soda [2]. These rules are auditable and cheap to run, but they scale poorly with schema complexity, detect only violations the author anticipated, and miss subtle multi-column or distributional anomalies. To address these limits, a recent line of work has proposed to complement or replace such rules with machine learning anomaly detectors embedded directly inside the ETL flow [3]–[5]. The existing literature, however, suffers from three gaps.

First, the pipelines I reviewed are mostly presented as architectures. Gopa [3] reports aggregate gains taken from other implementations and releases no code, and the abstract of Marupaka and Rangineni [4] announces accuracy gains without naming a dataset or a baseline. When an experiment is reported, it exercises a single algorithm on a single dataset: Sokkula and Vuppala [5] run Isolation Forest on Credit Card Fraud after oversampling the training data with SMOTE, and report an ROC-AUC of 0.51 with a fraud recall of 0.02. A near-random score on one dataset offers limited insight into when ML-based quality control is actually worth adopting.

Second, the anomaly detection community has produced excellent generic benchmarks—ADBench [6] compares 30 algorithms on 57 tabular datasets; PyOD [7] provides a unified implementation of a wide range of detectors—but these benchmarks evaluate algorithms on their own, outside of an ETL context. ADBench reports training and inference times for each algorithm, but it has no declarative rule-based baseline

of the kind that dominates production systems, and it does not measure what a detector costs inside a running pipeline (latency, memory, operational overhead).

Third, benchmarks typically report a single aggregated score per dataset, hiding the fact that different families of detectors excel at different kinds of anomalies [8]. ADBench does look at anomaly types, but as geometric types generated on synthetic data (local, global, dependency and clustered anomalies), not as the data quality faults an ETL pipeline meets. A Z-score detector is essentially unable to flag categorical format violations; Great Expectations trivially catches null-rate spikes but ignores multivariate outliers; autoencoders learn joint distributions but can be fooled by concept drift. A practitioner choosing a detector today lacks a fine-grained, actionable map from anomaly type to recommended method.

a) Contributions.: This paper addresses these three gaps. I present:

1) **A reproducible ETL pipeline** in which rule-based and classical ML anomaly detectors share one interface and are scored on the same extract–transform–detect path, released as open-source code. The registry holds three rule-based detectors, eight classical models and six neural ones, all behind the same interface.
2) **A head-to-head benchmark** of statistical rules (Z-score, IQR, Great Expectations), classical ML detectors (Isolation Forest, LOF, One-Class SVM, ECOD, COPOD, HBOS, KNN, PCA) and neural detectors (autoencoder, VAE, Deep SVDD, DIF, AE1SVM, LUNAR) on real and synthetic datasets, with a stratified held-out split, five seeds, and the ML operating point read from the data, reporting precision, recall, F1, ROC-AUC, PR-AUC, precision-recall curves, and detection-stage latency (fit, predict and score; extraction, transformation and loading are not timed).
3) **A per-anomaly-type analysis.** Using a synthetic dataset in which six distinct anomaly categories (missing values, duplicates, univariate outliers, multivariate outliers, temporal drift, invalid formats) are injected at a tunable severity, I produce a method-by-type recall and precision matrix, test its sensitivity under a severity sweep, add the neural models and the further classical detectors, and repeat it on two further schemas. The families turn out to have narrow, largely non-overlapping competence that the aggregate metrics hide.
4) **The same analysis on real data.** The six types are injected into 42 real tables of a pool of 152 open datasets, and the detectors are also run on its 100 labelled problems. The synthetic pattern holds only in part, and no detector is clearly ahead on the labelled problems.

The rest of the paper is organized as follows. Section II reviews related work on data quality in ETL and on anomaly detection benchmarks. Section III describes my pipeline architecture, the detectors I compare, the datasets, the metrics and the verification of the code. Section IV reports the main results, on three example datasets taken from the real-data pool, the synthetic generators and their variants, and the whole pool. Section V reports four secondary studies (a chronological split, decisions under temporal shift, an LSTM autoencoder and the cost of detection). Section VI discusses when rules, classical ML and deep learning are each the right tool and what limits the evidence, and Section VII concludes. Appendix A and B describe the verification of the code and its outcomes.

II. Related Work

a) Data quality in ETL pipelines..: Data quality is usually framed as governance: Khatri and Brown [1] treat data as an asset to be governed, and recent surveys list the tools that check it, including Great Expectations [9], Deequ and Soda among 17 tools [2], next to a broader survey of pipeline tools [10]. These tools are rule-based. Deequ expresses constraints as unit tests for data that run as aggregation queries on Spark, supports incremental validation, and also suggests constraints and detects anomalies in the time series of quality metrics [11]. They check what the author of the rules thought of in advance; they do not say how a learned detector would have done on the same data.

b) Anomaly detection algorithms.: The detectors I compare are standard. Isolation Forest [12], LOF [13] and the One-Class SVM [14] are available in PyOD [7], and Chandola et al. [15] survey the field. Goldstein and Uchida [8] compare 19 unsupervised algorithms on 10 datasets and find that the choice depends on the type of anomaly, local or global, which is the observation this paper takes to data quality faults. For reconstruction models, Malhotra et al. [16] train an LSTM encoder-decoder on normal behaviour and flag high reconstruction error, on public time series (power demand, space shuttle, ECG) and on engine sensors; my LSTM autoencoder follows that idea on windows of tabular rows.

c) Benchmarks.: ADBench [6] is the closest work: 30 algorithms on 57 datasets, 98,436 experiments, with different levels of supervision and four types of anomalies (local, global, dependency and clustered) generated from existing data, reported with AUCROC and AUCPR and with training and inference time. In its text I found no threshold-dependent metric and no chronological protocol, and none of its anomaly types is a data-quality fault such as a missing value or an invalid format. Agyemang [17] compares five unsupervised detectors on one simulated dataset with accuracy, precision, recall and F_1, and leaves real data as future work (I read the abstract only). AnoShift [18] builds a time-ordered protocol on ten years of Kyoto-2006+ network traffic, training on 2006–2010 and testing on 2011–2013 (NEAR) and 2014–2015 (FAR), and reports ROC-AUC and PR-AUC for classical and deep models, again without a decision threshold. FedADBench [19] compares deep models in a federated setting on four tabular datasets and chooses each threshold by maximising F_1 on a validation set with the anomaly ratio of the test set. Sun et al. [20] propose online thresholds with confidence sequences that adapt to distribution shift, and test them on synthetic and real data. They study how to pick a threshold, not how classical detectors behave in a pipeline.

d) Machine learning in ETL pipelines.: Papers on machine learning inside ETL mostly describe architectures. Gopa [3] reports aggregate figures taken from enterprise deployments without code, and Marupaka and Rangineni [4] announce accuracy gains in an abstract that names no dataset or baseline. Sokkula and Vuppala [5] run an Isolation Forest on the resampled Credit Card data and obtain an AUC-ROC of 0.51.

e) Gap and what is not claimed.: Among these sources I did not find a comparison of rule-based, classical and learned detectors that reports their decisions, not only a ranking, broken down by the kind of data-quality fault, on both a controlled generator and real tables, and in the setting of an ETL pipeline with its cost. That is the gap this paper addresses, with the limits stated in Section VI. I do not claim that no such study exists: the scan covers the works above, was not systematic, and I read the full text of only some of them. I do not claim that the detectors are new, that the injected faults match natural ones, or that the results hold beyond the datasets used.

III. Methods

A. Pipeline

Detectors are evaluated inside a minimal Extract–Transform–Load pipeline rather than in isolation. The extract stage reads the raw table, the transform stage removes exact duplicate rows, and the quality stage applies a detector that assigns each row a boolean anomaly flag and a continuous anomaly score. All detectors share a common `fit`/`predict`/`score` interface modelled on PyOD [7], so that classical and deep models can later be dropped in without changing the harness.

B. Rule-based detectors

This stage compares three rule-based baselines. All three define a band of "normal" values per column and flag a row when it leaves the band on at least one column; they differ in how the band is estimated and in how robust that estimate is to the very outliers being sought.

Z-score standardizes each numeric column on the fitted data and flags a row when any column exceeds $|z| > 3$. The threshold follows the empirical rule: under an approximately normal distribution about 99.7% of values lie within 3σ, so beyond it is rare by construction. Its weakness is that the mean and standard deviation are themselves inflated by extreme values, so a large outlier can mask others.

IQR learns Tukey's fences $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$ per column, where the interquartile range spans the central 50% of the data, and flags a row falling outside any fence. Quartiles are robust: unlike the mean and standard deviation they barely move when extreme values are added, so the fences are not distorted by the outliers they are meant to catch. The 1.5 multiplier corresponds to about 2.7σ under normality, isolating a comparable tail.

Great Expectations encodes a declarative expectation suite [2] and makes no distributional assumption: a not-null expectation on every column and a range expectation per numeric column, with bounds set to the 0.1th and 99.9th percentiles of the fitted data. Using inner percentiles rather than the observed min/max is what turns the range check into a usable rule instead of a no-op on the fitting data. A row is flagged when it violates at least one expectation; its score is the fraction of the suite it violates.

The continuous scores of the statistical detectors are the worst standardized deviation (Z-score) and the largest fence exceedance in IQR units (IQR). The axis of difference between the three is thus robustness and the assumption made about the shape of the data, which is precisely what the learned models of the next stage are meant to handle better.

C. Classical machine-learning detectors

The second family replaces the per-column band with a learned model of the data, so that a row can be flagged for an implausible *combination* of otherwise-normal values. Eight unsupervised detectors are taken from PyOD [7] rather than reimplemented, with two deviations explained below. All receive the same preparation: non-numeric columns are dropped, missing cells are median-imputed, and every feature is standardized, since several of them are distance-based. The PyOD `decision_function` provides the continuous score; the boolean flag uses a *contamination* equal to the anomaly prevalence of the training split (clamped to PyOD's valid range), so each model marks about as many rows as the data actually contains rather than a fixed share. Contamination sets only the flag threshold and leaves ROC-AUC and PR-AUC unaffected. All hyperparameters are the defaults of PyOD; I tuned none of them.

Three of the eight are the usual references. *Isolation Forest* [12] scores a row by how few random axis-aligned splits are needed to isolate it; *Local Outlier Factor* (LOF) [13] compares a point's local density to that of its neighbours; and a *One-Class SVM* [14] fits a boundary enclosing the bulk of the training data.

Five more complete the family, and they differ mainly in whether they look at the columns one by one or together. Three of them model each column on its own and add up the evidence, so they can flag an unusual value in a column but not an unusual pair of values. *ECOD* [21] estimates, for each column, the share of fitted rows below a value and the share above it (the empirical tail probabilities); a row scores the sum over columns of the negative logarithm of the smaller tail, the tail being chosen by the sign of the skewness of the column. *COPOD* [22] uses the same tail probabilities but combines the two tails of a column by their mean, which is an empirical copula without dependence between columns. *HBOS* [23] builds a histogram of each column and scores a row by the sum of the logarithms of the inverse heights of the bins its values fall in. The two others use the joint distribution. *KNN* [24] scores a row by its distance to its fifth nearest neighbour among the fitted rows, the simplest distance-based detector and the one LOF refines. *PCA* projects the rows onto the principal components that explain 95% of the variance

TABLE I
WHAT EACH WORK REPORTS, AS FAR AS I COULD READ IN THE CITED SOURCES. "NOT FOUND" MEANS ABSENT FROM THE TEXT I READ, NOT PROVEN ABSENT.

Work	Detectors and data	Time order	Metric at a threshold	Per-type results
ADBench [6]	30 algorithms, 57 datasets	not found	not found (AUCROC, AUCPR)	four geometric types, generated
Agyemang [17]	5 detectors, one simulated set	none	accuracy, precision, recall, F_1	not stated in the abstract
AnoShift [18]	classical and deep, Kyoto-2006+	IID, NEAR, FAR splits	not found (ROC-AUC, PR-AUC)	none
FedAD-Bench [19]	deep models, federated, 4 datasets	not found	F_1-optimal threshold on a validation set	none
Sun et al. [20]	thresholding method, synthetic and real	shift in the stream	error-rate guarantees	none
This paper	6 detectors (and 2 deep), 3 real sets, a pool of 152, synthetic and typed injection	chronological pilot and decision study on two datasets	precision, recall, F_1 at the training-prevalence threshold	six data-quality fault types

and scores the squared reconstruction error: a row that follows the correlations of the fitted data is rebuilt almost exactly, one that breaks them is not, while a shift that stays inside the kept components is invisible to it. The idea of detecting outliers with principal components is that of Shyu et al. [25].

Two choices differ from what PyOD does by default, because a detector in my pipeline must map a row to a score using the fitted rows only. PyOD recomputes the empirical distribution of ECOD and COPOD on the fitted rows plus the rows being scored, so a row's score depends on the other rows evaluated with it. I recompute the two methods on the fitted rows alone, ranking a row as if it were added to them, which is what PyOD computes for a batch of one row (a test compares the two). PyOD's own PCA scores a weighted distance to the eigenvectors, not a reconstruction error, so the projection comes from scikit-learn. LOF and KNN compare every row with the fitted rows, and the One-Class SVM is quadratic too; they are skipped, and the skip recorded, above $50{,}000$ fitted rows for LOF and KNN and $20{,}000$ for the One-Class SVM (limits set from the cost study).

D. Deep detectors

The third family learns a representation with a neural network. They are opt-in in the registry, they need PyTorch, and they run on the CPU. Six models are compared, all behind the same interface and all with the default settings of PyOD 2 [26]. The *autoencoder* is a multilayer perceptron (scikit-learn) trained to rebuild the standardized features through a bottleneck; a row that breaks the structure learned on the fitted data is rebuilt badly, and the squared reconstruction error is its score. The *variational autoencoder* (VAE) [27] encodes a row into a Gaussian code and decodes it, and is trained on a bound of the likelihood of the data; its score is the reconstruction error. PyOD draws the code at random when it scores, so a row's score changes from one call to the next and with the rows around it; I score with the mean of the code instead. *Deep SVDD* [28] trains a network to map the fitted rows close to a fixed centre and scores a row by its distance to that centre. The *Deep Isolation Forest* (DIF) [29] maps the rows through an ensemble of randomly initialized networks and builds an isolation forest in each representation; PyOD standardizes the representation of the scored rows with their own mean and variance, which makes a score depend on the other scored rows, so I standardize it with the fitted rows. *AE1SVM* [30] trains an autoencoder and a one-class SVM approximated with random Fourier features on its code, and scores the combination. *LUNAR* [31] learns the score from the distances of a row to its nearest neighbours, which generalizes LOF and KNN with a network. The neural models are fitted on at most $20{,}000$ randomly drawn fitted rows to bound the duration, and each is seeded with the seed of the repetition. The LSTM autoencoder of Section V-C reads windows of ordered rows, does not share the row-level contract, and stays outside this registry.

E. Synthetic per-anomaly-type dataset

The real datasets carry a single binary label that mixes every kind of defect together, so they cannot say *which* problem a detector is good at. I therefore generate a controlled dataset of $12{,}360$ customer records with Faker and NumPy, in which income is correlated with balance and transaction size and account age is spread over three years. Into this clean population I inject six anomaly types, each at a rate of 3% and each row tagged with its category: *missing values* (one or two cells nulled), *exact duplicates* (a clean row copied), *univariate outliers* (one numeric field pushed far out of range), *multivariate outliers* (every field individually in range but the combination implausible, e.g. an 18-year-old with a top-percentile balance and income), *temporal drift* (recent-signup rows whose income and balance distribution has shifted upward by half), and *invalid formats* (malformed e-mail, out-of-domain country code, impossible age). Because the category of every injected row is known, I report, per method and per type, both the detection rate (recall) and the precision on a stream made of clean rows plus that one type. The injection magnitudes are exposed as a single *severity* multiplier so the choice of magnitude can be swept rather than fixed (Section IV-H). I also add a time-aware detector,

drift, that learns a per-feature baseline from the oldest rows and flags recent rows that both sit in a window whose mean has moved and lean in the direction of that move; it is run on the synthetic dataset only, as the other detectors ignore the row order.

F. Datasets

The real data is one pool of 152 open datasets from ADBench and the UCI repository (open licence, pinned by checksum; `verification/dataset_pool.md`). I use it in two ways: 100 of its problems already carry labelled anomalies and are run as they are (Section IV-J), and 42 of its tables are large enough (3,000 rows, four continuous columns) to receive injected anomalies (Section IV-I). Three datasets are worked out in detail as examples, because they illustrate three regimes of the method. *Credit Card Fraud* [32] (Kaggle, also one of the ADBench problems; 283,726 transactions after de-duplication, 30 numeric features, 0.17% labelled fraud) is the standard strongly imbalanced benchmark, but its positive class mixes every kind of defect. *Cardio* (a cardiotocography anomaly dataset as packaged for outlier-detection benchmarking, another ADBench problem; 1,831 rows, 21 standardized features, 9.6% anomalies) is a *genuinely imbalanced* real dataset [6], [33]: the anomalies are rare and distributionally distinct, the regime the unsupervised detectors are built for. *Water Quality* [34] (a Kaggle dataset of the same kind, outside the pool; 3,276 samples, 9 numeric features) is the opposite case: the target class ("not potable") is common (61%) rather than rare, and the data has genuine missing values, so it probes the detectors outside their regime. Beside the real data, a *synthetic* dataset generated with Faker (12,360 rows, 17.5% anomalous, described above) carries the per-type labels, and two further generators repeat it on other schemas (Section IV-K), to test particular cases in controlled numbers.

G. Evaluation protocol

Each dataset is split 70/15/15 (train/validation/test), stratified on the label. For the synthetic dataset, rows are grouped by customer identity before stratification so an original and its exact copy cannot cross partitions. Detectors are fitted on the train split and every reported number is computed on the held-out test split; the validation split is reserved for later tuning. The ML contamination is read from the train split's prevalence. This is a label-informed operating-point choice, although estimator fitting itself does not use the labels. The real-data benchmark also permits a contamination value specified before evaluation, independent of every split's labels. This label-free preset forms a distinct comparison protocol; the numerical results in this manuscript retain the train-prevalence baseline. In the sensitivity sweep, contamination follows a fixed grid and no value is selected from the test metrics. Threshold or hyperparameter selection for a later confirmatory experiment would use training or validation data only. The whole procedure is repeated over five random seeds and I report the mean and standard deviation. This spread measures the sensitivity to the split and to model randomness on a fixed dataset; it is not a confidence interval for performance on new data. LOF and the One-Class SVM are run on Water Quality, Cardio and the synthetic dataset but not on Credit Card Fraud, where LOF's superlinear scoring and the One-Class SVM's quadratic training were expected to be too costly at 284k rows; this was not measured.

Because the detectors emit both a threshold decision and a graded score, I report threshold metrics (precision, recall, F_1) alongside ROC-AUC and PR-AUC (computed as average precision, not trapezoidal area). On imbalanced data PR-AUC is the more informative metric: the false-positive rate underlying ROC-AUC is diluted by the large true-negative pool and stays optimistic, whereas precision reacts directly to false alarms. I keep both because PR-AUC's random baseline equals the prevalence and is not comparable across datasets, while ROC-AUC's is always 0.5 and is. I also report the fraction of rows flagged and the in-pipeline latency. Because a single flag threshold hides the rest of the trade-off, I additionally draw the full precision-recall curve of every detector (Fig. 2) and sweep Isolation Forest's contamination across each dataset (Table VI). The deep-learning detectors (Autoencoder, LSTM-Autoencoder) are the object of the next stage.

H. Verification and reproducibility

The code that produces the results is checked at several levels, described in Appendix A. A routine suite exercises numerical contracts on small generated tables; a mutation campaign and twenty-three deliberate scientific defects ask whether those tests can fail; a real-data check compares the three research files with an archived manifest; and a release script turns the evidence into a verdict (*certified*, *incomplete* or *rejected*) tied to the source revision and to a fingerprint of the code that computes results. The verdict states where the evidence was produced and never that it was independently attested: I wrote the code and launched every run. A failed or missing check is a limitation of the evidence, not a successful experiment. The measured outcomes are in Appendix B. The chronological split and the window-based drift monitor have a separate protocol: their alert and delay measures are not pooled with the row-level metrics of the stratified benchmark.

IV. Results

Benchmark tables report means over five seeds on the held-out test split (Section III-G). Seed variability is reported in the archived result tables; timing also depends on the execution environment.

A. Example 1: Credit Card Fraud

Table II reports the detectors on Credit Card Fraud: the eight that scale to 284k rows, then the six neural ones, which are fitted on at most 20,000 rows. The statistical rules recover most frauds (recall 0.87–0.97) at a precision below 0.04: a fixed per-column rule cannot separate a 0.17% positive class from the tails of legitimate traffic. Great Expectations is the most selective of the rules (3% of rows) and has the best PR-AUC of the three (0.182 ± 0.046). Isolation Forest behaves differently

TABLE II
DETECTORS ON CREDIT CARD FRAUD, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. HIGHER IS BETTER EXCEPT FLAGGED ROWS AND TIME.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%	Time (s)
Z-score	0.011	0.870	0.021	0.926	0.078	13.4	**0.03**
IQR	0.003	**0.966**	0.007	0.911	0.018	48.6	0.12
Great Expectations	0.037	0.676	0.070	0.830	0.182	3.1	1.26
Isolation Forest	0.202	0.206	0.203	0.934	0.120	0.2	1.70
ECOD	0.260	0.268	0.262	0.930	0.175	0.2	3.04
COPOD	0.310	0.299	0.303	0.927	0.213	0.2	3.04
HBOS	**0.337**	0.318	0.326	0.931	0.216	0.2	0.97
PCA	0.139	0.132	0.135	0.831	0.105	0.2	0.70
Autoencoder	0.083	0.141	0.103	0.892	0.066	0.3	–
VAE	0.162	0.237	0.191	0.929	0.142	0.2	–
Deep SVDD	0.261	0.546	**0.346**	0.918	**0.289**	0.4	–
DIF	0.287	0.355	0.317	0.922	0.242	0.2	–
AE1SVM	0.184	0.223	0.201	0.937	0.144	0.2	–
LUNAR	0.138	0.535	0.216	**0.941**	0.136	0.7	–

now that its contamination matches the prevalence: it flags only 0.17% of rows and reaches a precision of 0.20, the first usable operating point among the rules and Isolation Forest, but its recall drops to 0.21 and both are seed-sensitive ($F_1 = 0.20 \pm 0.07$). The column-wise detectors added to the registry do better than Isolation Forest on this dataset: with the same 0.2% of rows flagged, HBOS reaches the best precision of the table (0.34 ± 0.05), COPOD and ECOD an F_1 of 0.30 and 0.26 against 0.20, and the PR-AUC of HBOS and COPOD (0.22 and 0.21) is above that of Great Expectations. PCA is below Isolation Forest ($F_1 = 0.14 \pm 0.14$). Among the neural models, Deep SVDD has the best F_1 (0.35 ± 0.08) and PR-AUC (0.29 ± 0.10) of the table, at a recall of 0.55, and LUNAR the best ROC-AUC (0.941) with a precision of only 0.14 and 0.7% of rows flagged; the autoencoder is the weakest ($F_1 = 0.10$). Seed spreads are large for every learned model, so the order among the better ones is not firm. Great Expectations took 1.3 s on this run, against 0.03–0.12 s for the statistical rules; these times vary between runs, and the neural models were timed under load, so their column is left empty (Section V-D measures them alone). LOF, KNN and the One-Class SVM were not run at 284k rows, where their cost was expected to be prohibitive; I did not measure it.

B. Example 2: Cardio, rare and distinct anomalies

Table III runs all detectors on Cardio, where the anomalies are both rare (9.6%) and distributionally distinct. Here the unsupervised detectors work: Isolation Forest reaches $F_1 = 0.62$, PR-AUC 0.70 and ROC-AUC 0.94, and the One-Class SVM is close behind. Z-score also does well on this dataset ($F_1 = 0.61$) because the 21 features are already standardized and the anomalies stand out on several of them at once. ECOD and COPOD reach the level of Isolation Forest (PR-AUC 0.69 for both against 0.70, F_1 0.57 and 0.60), while HBOS ($F_1 = 0.51$), KNN (0.43) and PCA (0.36) fall below it. Among the neural models, DIF ($F_1 = 0.59$, PR-AUC 0.67) and AE1SVM (0.61, 0.64) come close to Isolation Forest, whereas the autoencoder, Deep SVDD and LUNAR are far

TABLE III
ALL DETECTORS ON CARDIO, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. 1,831 RAW ROWS (1,822 AFTER DEDUPLICATION), 9.6% ANOMALIES.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%
Z-score	0.49	0.81	0.61	0.93	0.61	16.6
IQR	0.21	**0.90**	0.33	0.78	0.30	43.4
Great Expectations	0.50	0.21	0.29	0.59	0.19	4.6
Isolation Forest	**0.64**	0.61	**0.62**	0.94	**0.70**	9.9
LOF	0.25	0.27	0.26	0.62	0.24	10.7
One-Class SVM	0.56	0.61	0.58	0.94	0.63	10.7
ECOD	0.57	0.59	0.57	**0.95**	0.69	10.2
COPOD	0.61	0.61	0.60	0.93	0.69	9.9
HBOS	0.52	0.52	0.51	0.86	0.57	10.2
KNN	0.40	0.47	0.43	0.78	0.47	11.7
PCA	0.33	0.39	0.36	0.73	0.37	11.4
Autoencoder	0.32	0.38	0.35	0.74	0.35	11.7
VAE	0.41	0.46	0.43	0.87	0.45	11.2
Deep SVDD	0.26	0.41	0.32	0.69	0.29	15.8
DIF	0.53	0.69	0.59	0.94	0.67	13.1
AE1SVM	0.58	0.64	0.61	0.94	0.64	10.9
LUNAR	0.41	0.39	0.29	0.65	0.33	18.8

TABLE IV
ALL DETECTORS ON THE SYNTHETIC DATASET, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TIME IS THE IN-PIPELINE FIT+PREDICT+SCORE LATENCY.

Detector	Prec.	Rec.	F_1	ROC	PR	Time (s)
Z-score	**1.000**	0.196	0.328	**0.742**	**0.544**	**0.004**
IQR	0.584	0.396	0.472	0.675	0.452	0.007
Great Expectations	0.951	0.202	0.333	0.600	0.333	0.46
Isolation Forest	0.495	**0.499**	**0.497**	0.735	0.485	0.34
LOF	0.388	0.420	0.404	0.659	0.437	0.22
One-Class SVM	0.478	0.485	0.481	0.722	0.520	4.20

below (F_1 between 0.29 and 0.35). This is the counterpoint to Credit Card Fraud and Water Quality: the value of a learned detector shows when the anomalies are genuinely rare outliers, not when the positive class is a 0.17% mixed-defect label or a 61% majority. Isolation Forest and the One-Class SVM carry a wide seed spread on this small dataset (F_1 standard deviations of 0.14 and 0.17).

C. Synthetic dataset: aggregate metrics

Table IV reports the six core detectors on the synthetic dataset. Aggregate metrics still compress the families together: F_1 ranges from 0.33 to 0.50 and PR-AUC from 0.33 to 0.54. Isolation Forest has the best F_1 (0.50) and Z-score keeps a perfect precision by firing only on the most extreme rows. The One-Class SVM approaches Isolation Forest on quality while taking longer in this run. Read alone, these numbers suggest the detector choice barely matters; the per-type breakdown shows the opposite.

D. Synthetic dataset: by anomaly type

Fig. 1 gives the mean recall of each method of the registry on each injected type, with the LSTM autoencoder of Section V-C for reference. The structure the aggregate table hides is now clear: most methods have a narrow competence,

and the neural models and the new classical ones change the picture only on some columns.

- *Univariate outliers* are caught by every distributional method (recall 0.68–1.00) except Deep SVDD (0.33), and only by those. Z-score's precision on them is 1.00: it never fires on a clean row.
- *Missing values* are caught best by the Great Expectations not-null expectations (recall 1.00, precision 0.94), because the statistical rules propagate `NaN` to a non-flag and the learned models see imputed values. LUNAR (0.44) and LOF (0.28) are the next best, and every other model stays below 0.20.
- *Exact duplicates* are nearly invisible to every detector (recall at most 0.18, except LUNAR with 0.39). The deduplication rule of the transform stage is the only thing that catches them, at precision 0.50 because it also flags each row's clean original. In this grouped split the rule is evaluated within the held-out partition; it is a stream-level reference, not a model.
- *Multivariate outliers* separate the families, and here the new detectors matter. Isolation Forest (0.89) and the One-Class SVM (0.90) were the only methods with real recall among the first six, since they are the only ones that model the joint distribution. Among the neural models, DIF (1.00), the VAE (0.92), the autoencoder (0.83) and AE1SVM (0.78) do as well, and LUNAR (0.63) follows; Deep SVDD (0.13) does not. KNN (0.39) and PCA (0.48) are below the forest. ECOD and COPOD, which look at each column alone, still reach 0.52 and 0.51, which I had not expected; probably the injected combination also puts one of the values in a tail of its own column. HBOS, built on histograms of single columns, stays at 0. The precision of Isolation Forest and the One-Class SVM on this type is only about 0.22: they cast a wide net that also catches clean rows.
- *Invalid formats* are caught mainly through the impossible numeric value they carry (IQR recall 1.00, precision 0.37; Z-score precision 1.00; DIF 1.00, LUNAR 0.90, KNN 0.81); the malformed string fields are missed by all.
- *Temporal drift* is the weakest column for most methods. LUNAR is the best (0.65), ahead of LOF (0.50), the best of the first six; every other method stays below 0.42. The dedicated *drift* detector scores 0 here too: at the reference magnitude the shifted values stay inside the global range, and once diluted by the many clean recent rows the window mean has moved by less than a tenth of a standard deviation on average (0.04 to 0.11 over the five test splits). The present row-level drift detector remains at zero even in the severity sweep; the windowed-monitoring extension is separate.

The practical reading is a short decision list: null-rate problems to declarative rules, single-column outliers to any statistical rule, implausible combinations to a tree-, boundary- or reconstruction-based model (Isolation Forest, the One-Class SVM, DIF, the VAE), and duplicates and drift to dedicated stages the detectors do not replace, with LUNAR as the one learned model that helps on drift at the price of more false alarms (Section IV-I).

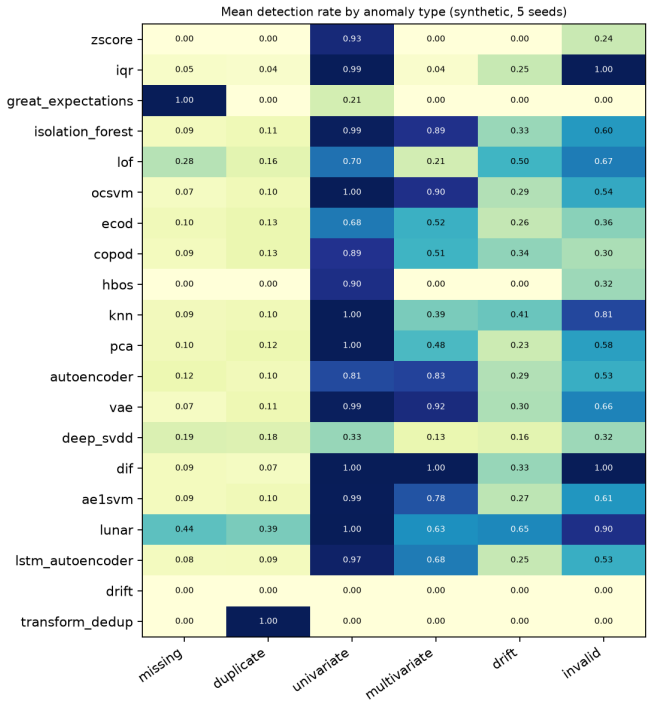


Fig. 1. Mean detection rate (recall) over five seeds of each method on each injected anomaly type, synthetic test split. Darker is higher. `transform_dedup` is the deduplication rule of the transform stage, shown for reference.

E. Example 3: Water Quality, a common target class

Table V runs all detectors, the neural ones included, on Water Quality, where "not potable" is the target and covers 61% of the rows. Every detector's ROC-AUC sits between 0.45 and 0.51 and every PR-AUC lies within 0.05 of the 0.61 base rate: the unsupervised detectors are essentially at chance. This is the expected outcome and a useful negative result. When the target class is common and not distributionally distinct from the rest, "anomaly" detection has nothing to key on. The dataset still exercises the pipeline on real missing data: Great Expectations flags 40% of rows on its not-null expectations, but the nulls here are uncorrelated with potability, so that recall buys no precision.

F. Precision-recall curves and the operating point

Fig. 2 draws the full precision-recall curve of every detector on each dataset's test split. The curves make the aggregate tables concrete: on Cardio the Isolation Forest and One-Class SVM curves stay high (AP 0.80 and 0.76); on Water Quality every curve hugs the 0.61 base-rate line; on Credit Card Fraud Great Expectations and Isolation Forest dominate but neither clears a precision of 0.4 except at negligible recall. Table VI makes the point numerically for Isolation Forest: as the contamination moves from 0.001 to 0.20, its Credit Card

TABLE V

ALL DETECTORS ON WATER QUALITY, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TARGET: "NOT POTABLE", BASE RATE 0.61.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%
Z-score	0.41	0.02	0.05	0.46	0.58	3.5
IQR	0.57	0.09	0.16	0.49	0.60	10.0
Great Expectations	**0.62**	0.41	0.49	**0.51**	**0.61**	40.1
Isolation Forest	0.57	0.46	0.51	0.45	0.57	49.7
LOF	0.58	0.46	0.52	0.46	0.58	48.7
One-Class SVM	0.57	0.45	0.50	0.46	0.57	48.4
ECOD	0.58	0.47	0.52	0.45	0.57	49.6
COPOD	0.58	0.48	0.52	0.46	0.57	50.3
HBOS	0.58	0.47	0.52	0.46	0.58	49.8
KNN	0.57	0.46	0.51	0.45	0.57	48.4
PCA	0.59	0.48	0.53	0.47	0.59	49.8
Autoencoder	0.60	0.49	0.54	0.47	0.58	50.0
VAE	0.58	0.48	0.53	0.46	0.58	50.7
Deep SVDD	0.61	0.52	**0.56**	0.49	0.60	52.4
DIF	0.58	0.48	0.53	0.46	0.58	51.1
AE1SVM	0.56	0.45	0.50	0.45	0.57	49.3
LUNAR	0.58	**0.53**	0.55	0.45	0.57	55.4

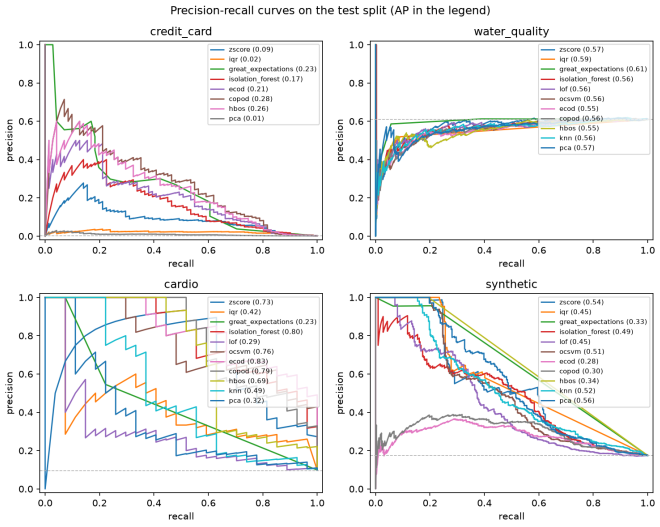


Fig. 2. Precision-recall curves on the test split, one panel per dataset, average precision in the legend. The dashed line is the base rate.

Fraud precision falls from 0.37 to 0.008 while recall rises from 0.23 to 0.92; on Cardio the same sweep trades a precision of 1.00 at recall 0.04 for a recall of 0.93 at precision 0.46. The estimator is fixed; only the threshold moves. Reading the prevalence off the training split lands near the knee of the curve on the datasets where a knee exists.

G. Fixed-contamination sensitivity analysis

The train-prevalence operating point uses labels from the training partition. To measure its influence, I reran the three example datasets with the same five seeds and partitions, replacing that value by a constant contamination of 0.02 for the ML detectors. Table VII shows Isolation Forest, the only learned detector run on all three datasets. The ranking score and its average precision are unchanged; only the binary

TABLE VI

ISOLATION FOREST UNDER A CONTAMINATION SWEEP, TEST SPLIT, ONE SEED. "PREV." IS THE TRAINING-SPLIT PREVALENCE, THE VALUE THE BENCHMARKS USE.

Dataset	Contam.	Prec.	Rec.	Flag.%
Credit Card Fraud	0.001	0.37	0.23	0.10
Credit Card Fraud	0.10	0.01	0.90	10.2
Credit Card Fraud	prev. 0.002	0.28	0.32	0.20
Cardio	0.01	1.00	0.33	3.3
Cardio	0.20	0.46	0.93	19.7
Cardio	prev. 0.096	0.72	0.67	9.1
Water Quality	prev. 0.5	0.58	0.49	51.6
Synthetic	prev. 0.175	0.53	0.51	16.8

TABLE VII

ISOLATION FOREST WITH TRAIN-PREVALENCE AND FIXED 0.02 CONTAMINATION, SAME FIVE REAL-DATA SPLITS. ENTRIES ARE MEANS; "FLAG" IS THE FRACTION OF TEST ROWS PREDICTED ANOMALOUS.

Dataset	F_1 train	F_1 fixed	Flag train%	Flag fixed%
Credit Card Fraud	0.203	0.102	0.168	2.063
Cardio	0.618	0.406	9.854	2.701
Water Quality	0.512	0.023	49.675	1.789

decision threshold moves. On Credit Card Fraud, the fixed setting raises recall from 0.206 to 0.682 but lowers precision from 0.202 to 0.055. On Cardio, it raises precision from 0.639 to 0.983 but lowers recall from 0.615 to 0.267. The Water Quality target is a majority class, so a 2% outlier budget leaves almost all positive rows unflagged. The fixed value was chosen before these comparator runs but after the baseline results were available. These numbers are therefore a post-hoc sensitivity analysis, not a new confirmatory estimate on an untouched test set.

H. Detection rate versus injection severity

The synthetic magnitudes are a design choice, so I sweep them. A single *severity* multiplier scales the univariate factor, the multivariate percentile extremity and the temporal-drift multiplier (1.0 is the dataset used above). Fig. 3 plots per-type recall against severity. Univariate outliers are already saturated at severity 0.5. Multivariate outliers show a threshold: Isolation Forest and the One-Class SVM climb to recall 1.0, while IQR jumps from 0.04 to 1.0 once the injected percentile crosses its fence. Temporal drift becomes easier for IQR, Isolation Forest and One-Class SVM (roughly 0.2–0.3 recall at severity 1.0 and 0.6–0.7 at severity 3), while LOF peaks earlier and then falls. The row-level drift detector does not respond in this sweep. The ranking therefore changes with severity, limiting claims based on one injection magnitude.

I. Typed injection on real datasets

The synthetic matrix of Section IV-D comes from a single generator. To see whether its pattern depends on that generator, the six anomaly types were injected into 42 real tables from the UCI repository (open licence, pinned by checksum; `verification/dataset_pool.md`). For each ta-

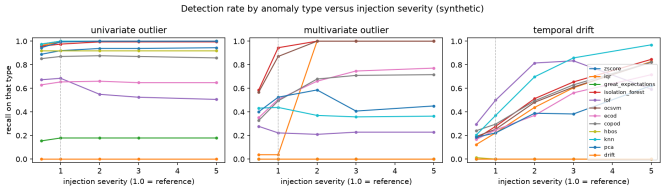


Fig. 3. Per-type recall against injection severity (1.0 is the reference synthetic dataset), mean over three seeds. The dashed line marks severity 1.0.

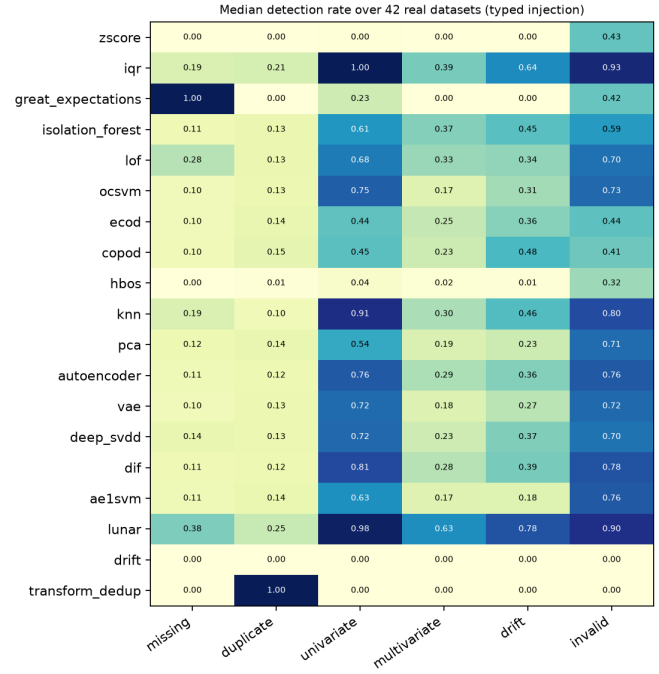


Fig. 4. Median detection rate over 42 real datasets with typed injection. Compare with Figure 1.

ble only the continuous columns are kept, and for labelled sets only the normal class, so that the original anomalies never enter the evaluation. The injection follows the synthetic one (three percent of rows per type, same split, seeds and detector settings), with numeric versions of the types: a value several standard deviations away, a pair of correlated columns pushed to opposite tails, a shift of three columns for the most recent rows, a sign flip, a sentinel or a thousandfold scale error. The protocol and five hypotheses were written before any run (`verification/typed_injection.md`).

Two rows of the synthetic pattern hold on every table: Great Expectations finds all missing values and the deduplication rule all duplicates. The rest does not carry over. Three of the five hypotheses are not supported (Great Expectations and deduplication aside, the specialisation, the multivariate gap and the difficulty of drift). On multivariate outliers the median detection rate is 0.37 for Isolation Forest and 0.17 for the one-class SVM, against 0.89 and 0.90 on the synthetic data. The interquartile fences reach the best detection rate on several types, but they also flag a median of 22% of the clean rows (up to 99% on some wide tables), so the detection rate alone favours them. The z-score detects almost nothing on univariate outliers (median 0.00 against 0.93 synthetically). A check made after seeing this, and therefore exploratory, shows why: the invalid rows of the training split inflate the standard deviation, and without them the z-score detects every univariate outlier on all 42 tables. The result says more about non-robust statistics on a contaminated training set than about the z-score itself.

The two hypotheses on stability hold. The mean rank correlation between the method-by-type matrices of two tables is 0.80 with the seventeen detectors of the registry (it was 0.83 with the first eight rows), and it is lowest for drift (0.63), multivariate outliers (0.65) and duplicates (0.66), highest for univariate outliers (0.76). Part of this stability is mechanical, since many cells are close to 0 or 1. The detection rate of Isolation Forest on univariate outliers falls with the number of continuous columns (rank correlation -0.96 over the 42 tables). The anomalies are injected, not natural, and at one severity.

The classical and neural detectors added to the registry were run on the same 42 tables (Figure 4), without a hypothesis of their own on this study. On multivariate outliers LUNAR has the highest median detection rate (0.63, Isolation Forest 0.37, KNN 0.30, the autoencoder 0.29, DIF 0.28, ECOD 0.25), and also on drift (0.78 against 0.64 for the IQR rule and 0.45 for Isolation Forest) and on missing values among the learned models (0.38, Great Expectations 1.00). It pays for it: it flags a median of 24% of the clean rows (97% on the worst table), where the other learned models, calibrated to the same training prevalence, flag 12% to 15% and HBOS 1%. Only the IQR rule and LUNAR flag more than a fifth of the clean rows on median. The column-wise ECOD and COPOD detect univariate outliers less often than KNN (0.44 and 0.45 against 0.91) and HBOS almost never (0.04). So the synthetic reading holds in outline: no detector covers every type, and the one that comes closest buys its recall with false alarms.

J. Labelled problems of the dataset pool

The three examples above are only a few datasets. The pool of 152 open tables (`verification/dataset_pool.md`), of which Credit Card Fraud and Cardio are members, holds 100 problems whose anomalies are already labelled: 36 from ADBench and 64 from the UCI repository, where the rarest class (at least 20 rows) is the anomaly and the other classes are normal; a class above 10% of the rows is resampled to 10%. I first ran the six detectors of the main benchmark on all of them with the protocol of Section III-G, written down before the run (`verification/labelled_pool.md`): one normalisation for every table, a stratified 70/15/15 split, five seeds, contamination equal to the training prevalence and default settings. I then added the five other classical detectors and the six neural ones of the registry on the same problems, with their own protocol and hypotheses (`verification/full_registry.md`); the neural

TABLE VIII
MEDIAN OVER THE LABELLED PROBLEMS OF THE SEED-AVERAGED TEST METRICS, WITH A 95% BOOTSTRAP INTERVAL OVER THE PROBLEMS, AND THE MEDIAN RANK OF THE METHOD ON PR-AUC AMONG THE SEVENTEEN (1 IS BEST). THE FIRST BLOCK IS THE RULES, THE SECOND THE CLASSICAL MODELS, THE THIRD THE NEURAL ONES.

Detector	Problems	PR-AUC	ROC-AUC	Rank
Z-score	100	0.203 [0.164, 0.279]	0.626	9.0
IQR	100	0.144 [0.117, 0.191]	0.556	12.0
Great Expectations	100	0.115 [0.105, 0.123]	0.508	16.0
Isolation Forest	100	0.215 [0.158, 0.293]	0.677	7.0
LOF	92	0.165 [0.153, 0.237]	0.588	10.0
One-Class SVM	84	0.240 [0.177, 0.307]	0.670	8.0
ECOD	100	0.240 [0.191, 0.305]	0.656	7.0
COPOD	100	0.216 [0.162, 0.314]	0.691	7.0
HBOS	100	0.237 [0.176, 0.311]	0.663	8.0
KNN	92	0.203 [0.172, 0.284]	0.658	8.0
PCA	100	0.178 [0.152, 0.219]	0.581	10.0
Autoencoder	100	0.205 [0.150, 0.240]	0.631	8.0
VAE	100	0.216 [0.163, 0.279]	0.676	7.0
Deep SVDD	100	0.181 [0.147, 0.226]	0.619	10.0
DIF	100	0.226 [0.173, 0.272]	0.639	8.0
AE1SVM	100	0.210 [0.172, 0.253]	0.635	8.0
LUNAR	100	0.183 [0.153, 0.225]	0.615	10.0

models are fitted on at most 20,000 rows. LOF and KNN are skipped above 50,000 training rows and One-Class SVM above 20,000, so they cover 92, 92 and 84 problems; the skips are listed in the archive. All runs completed: none of the six neural models failed on any of the 100 problems.

No detector is clearly ahead. ECOD and the One-Class SVM share the highest median PR-AUC (0.240), followed by HBOS (0.237) and, among the neural models, DIF (0.226); the intervals overlap broadly with each other and with the Z-score, so the hypothesis that no method is universal is supported on the first six detectors, and it also holds for the seventeen. The best method differs from one problem to the next: among the first six, Isolation Forest is first on 26 problems, LOF on 24, the IQR rule on 16, the Z-score on 14, One-Class SVM on 13 and Great Expectations on 7; among all seventeen, DIF is first on 13 problems, HBOS on 10, the IQR rule on 9, and every method is first on at least two. Learning does help on median: Isolation Forest is above the Z-score on 67 problems and above the IQR rule on 62, but the gain is small (about 0.01 to 0.04 of PR-AUC). The median ROC-AUC stays between 0.51 and 0.68, and on 30 problems no detector reaches 0.6. These anomalies are mostly not outlying in the sense these detectors assume, which is expected for labels taken from a rare class rather than from a data-quality defect. Great Expectations, built for declared constraints, is close to chance here.

Three hypotheses of the registry study concern this table. The first holds: ECOD, HBOS and COPOD have a higher median PR-AUC than Isolation Forest (0.240, 0.237 and 0.216 against 0.215), although the gap is small and KNN (0.203) and PCA (0.178) are below it. The second holds too: the best neural model (DIF, 0.226) is below the best classical one (0.240), so a neural model does not beat the classical ones by the margin of 0.02 that I had set. The third holds, for the reason given above: no neural model failed.

Two of the five hypotheses on the first six detectors are not supported. The advantage of Isolation Forest over the Z-score was expected to vanish when anomalies are more than 20% of the rows; it is smaller there (+0.013 of median PR-AUC on 12 problems) but still positive, as it is below 5% (+0.037, 19 problems) and in between (+0.009, 69 problems). The IQR rule was expected to flag more than three times the anomaly prevalence; its median is 2.3 times, the Z-score's 1.7, and the learned detectors flag about the prevalence by construction. The order of the methods is the same for ADBench and for UCI (rank correlation 0.94), although the level is much lower on UCI (best median PR-AUC 0.19 against 0.40). Nineteen sources appear in both lists with different derivations, so the problems are not independent, and 10% resampling fixes the median prevalence at 0.10. The ADBench half is closer to the usual benchmark and the UCI half is a harder, rule-derived labelling; I did not tune any detector, so the numbers describe default settings.

a) Two more examples from the pool.: The three examples of this section were picked for the regimes they show, and the pool holds two regimes they do not. Among the 100 problems, the largest margin of the rule-based detectors over the learned ones is on Annthyroid (7.6% anomalies): the IQR rule reaches a PR-AUC of 0.61 and the best learned model 0.31. The largest margin of a neural model over the other families is on Mushroom (10% anomalies, a class derived by a rule): LUNAR reaches 0.96 and the best classical model 0.63. I chose them after seeing the results, as illustrations, so this is an exploratory choice and not a test. Table IX lists the seventeen detectors on these two problems and on Credit Card Fraud and Cardio as they appear in the pool, normalised like the others, so their numbers differ slightly from the tables above. It shows the absence of a universal detector at the scale of a single dataset: the IQR rule is the best on Annthyroid and at chance on Mushroom (0.10), where LUNAR leads; Deep SVDD is first on Credit Card Fraud and Isolation Forest on Cardio.

K. Two further synthetic schemas

The synthetic matrix comes from one table of customers, so its pattern could come from that table. I wrote two more generators, a table of card transactions and a table of sensor readings, each with six numeric fields, a date, a string with a checkable format and a category with a finite domain, and injected the same six types at the same rate and the same protocol, with four hypotheses fixed beforehand (`verification/multi_schema.md`). The customer table and its published results are untouched; its matrix below is the published one.

The structural rows carry over exactly: Great Expectations finds every missing value and the deduplication rule every duplicate, on all three schemas. The overall profile is stable too: the rank correlation between the method-by-type matrices of two schemas is 0.93 on average (0.91 to 0.96) for the detectors of the first six rows, and 0.89 (0.86 to 0.92) when the

TABLE IX
PR-AUC OF THE SEVENTEEN DETECTORS ON FOUR PROBLEMS OF THE POOL (MEAN OVER FIVE SEEDS). CREDIT CARD FRAUD (0.2% ANOMALIES) AND CARDIO (9.6%) ARE THE EXAMPLES OF SECTION IV; ANNTHYROID AND MUSHROOM ARE THE TWO ADDITIONAL ONES. A DASH MEANS THE DETECTOR WAS SKIPPED ON THAT SIZE.

Detector	Fraud	Cardio	Annthyroid	Mushroom
Z-score	0.10	0.61	0.18	0.44
IQR	0.02	0.30	**0.61**	0.10
Great Expectations	0.22	0.19	0.10	0.12
Isolation Forest	0.13	**0.70**	0.29	0.63
LOF	–	0.24	0.21	0.26
One-Class SVM	–	0.63	0.19	0.40
ECOD	0.21	0.69	0.28	0.41
COPOD	0.26	0.69	0.18	0.44
HBOS	0.26	0.58	0.24	0.50
KNN	–	0.47	0.23	0.59
PCA	0.03	0.37	0.12	0.37
Autoencoder	0.11	0.35	0.14	0.68
VAE	0.10	0.45	0.18	0.49
Deep SVDD	**0.34**	0.29	0.23	0.40
DIF	0.27	0.67	0.23	0.45
AE1SVM	0.18	0.64	0.31	0.40
LUNAR	0.15	0.33	0.20	**0.96**

TABLE X
MEAN RECALL OVER FIVE SEEDS ON THE TWO FURTHER SCHEMAS FOR THE FIRST SIX METHODS THAT CARRY THE PATTERN (CUSTOMER TABLE IN FIGURE 1; THE FULL MATRICES WITH THE OTHER DETECTORS ARE IN APPENDIX C). COLUMNS: MISSING VALUES, DUPLICATES, UNIVARIATE OUTLIERS, MULTIVARIATE OUTLIERS, DRIFT, INVALID FORMATS.

Schema	Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Transactions	Great Expect.	1.00	0.00	0.14	0.00	0.00	0.00
	Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00
	IQR	0.09	0.14	0.98	0.57	0.37	0.63
	Isolation Forest	0.09	0.18	0.97	0.49	0.38	0.56
	One-Class SVM	0.10	0.26	0.89	0.18	0.36	0.39
Sensors	Great Expect.	1.00	0.00	0.23	0.00	0.00	0.00
	Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00
	IQR	0.03	0.05	1.00	0.02	0.94	1.00
	Isolation Forest	0.08	0.12	1.00	0.32	0.96	0.97
	One-Class SVM	0.08	0.10	1.00	0.33	0.94	1.00

new classical and the neural detectors are added. Two things do not carry over. For multivariate outliers, Isolation Forest and One-Class SVM reach 0.89 and 0.90 on the customers, but only 0.49 and 0.18 on the transactions and 0.32 and 0.33 on the sensors, and on the transactions the IQR rule reaches 0.57. For drift, the best recall is 0.50 on the customers and 0.38 on the transactions but 0.96 on the sensors, where a factor of 1.5 on a signal of about -70 dBm moves values several standard deviations away. The per-row drift detector of the registry, which was designed for dilute shifts, flags nothing on any schema. Two of the four hypotheses (structural rows, rank stability) hold; the multivariate gap and the weakness of drift do not. The new detectors do not remove this dependence. On multivariate outliers the best model is not the same on each schema: DIF reaches 1.00 on the customers and 0.99 on the sensors but only 0.37 on the transactions, where LUNAR (0.73) and Deep SVDD (0.70) do best. On drift, LUNAR and KNN reach 1.00 on the sensors, and LUNAR 0.65 and 0.55 on the customers and the transactions. The injected magnitudes are set in relative terms, so how visible drift and multivariate outliers are depends on how the column is scaled and correlated, not only on the type. The three schemas share an author and an injection logic, so this shows that the profile does not depend on the shape of one table, not that it holds on real data (Section IV-I).

L. Neural models and further classical detectors in the per-type matrix

The first six rows of Figure 1 have no learned reconstruction model. I added two on the same synthetic data and the same identity-grouped splits and seeds, with the protocol and four hypotheses written beforehand (`verification/deep_typed.md`): the sklearn autoencoder of the registry, and the LSTM autoencoder of Section V-C on windows of the ten preceding customers ordered by signup date. Unlike the Credit Card pilot, both models were trained until their validation loss stopped improving (patience of 5, at most 300 epochs), and all ten trainings stopped before the cap (60 to 178 epochs). The other rows were recomputed by the same code and equal the published matrix.

Those two rows do not change the picture. The autoencoder detects multivariate outliers (0.83 ± 0.10) about as well as Isolation Forest (0.89) and One-Class SVM (0.90), as expected for a model that reconstructs correlations, and the LSTM is a little lower (0.68 ± 0.12). Both are close to the learned detectors on the structural types, and on missing values and duplicates they are no better than the best row-level detector. The hypothesis I expected to matter, that the sequence model would catch temporal drift better than every row-level detector, is not supported: its recall is 0.25 ± 0.07 against 0.50 for LOF, 0.33 for Isolation Forest and 0.29 for the autoencoder. Three of the four hypotheses hold (structural types, convergence, multivariate outliers). The synthetic customers are independent rows, so the signup order carries a sequence only for the drifted rows, which are the most recent ones; a window of ten rows that are mostly clean and have shifted values in only some of the columns gives the LSTM little to learn from. This says that this construction of drift is not a sequence problem, not that sequence models cannot detect drift. With five seeds the standard deviations are indicative and none of these comparisons is a test.

The complete-registry study then added five classical detectors and five neural ones (Section III) to the same matrix, with three hypotheses on it written beforehand (`verification/full_registry.md`). None is supported. The detectors that read one column at a time were expected to miss multivariate outliers (recall at most 0.20 for ECOD, COPOD and HBOS, and Isolation Forest at least 0.50 above them): only HBOS does (0.00), while ECOD and COPOD reach 0.52 and 0.51. PCA was expected to match the autoencoder within ten points, since both rebuild the rows, and it does not (0.48 against 0.83): a reconstruction through 95% of the variance rebuilds a row that breaks the correlations only

when the broken direction is a minor one. No new detector was expected to beat the best earlier row-level detector on drift by more than ten points; LUNAR does (0.65 against 0.50 for LOF). Its other rows are also the highest of the matrix on missing values (0.44) and duplicates (0.39), but it flags 41% of the clean rows, against 9% to 16% for the other neural models, so its recall comes with a much higher cost in false alarms; the same holds on the real tables of Section IV-I. DIF reaches 1.00 on both univariate and multivariate outliers and on invalid formats, with 9% of clean rows flagged; the VAE (0.92) and AE1SVM (0.78) follow the autoencoder on multivariate outliers; Deep SVDD is the weakest neural model on univariate and multivariate outliers, drift and invalid formats.

M. Every study on the whole pool

The curves, the contamination sweep, the label-free threshold, the severity sweep, the pipeline run and the chronological split were first done on the three example datasets and the synthetic data. I applied each of them to every dataset of the pool for which it is defined, with the protocol and six hypotheses written beforehand (`verification/pool_extension.md`), and list what could not be run and why.

a) Label-free threshold and curves.: Table XI gives, for the seventeen detectors on the 100 labelled problems, the median F_1 and the median share of flagged rows at the training-prevalence threshold and at a constant contamination of 0.02, both read from a single fit (a test checks that this equals a refit). The hypothesis that the label-free threshold lowers the median F_1 of at least twelve of the seventeen detectors holds: it does for the 14 learned detectors, and the three rules are unchanged because they do not read the contamination. The ranking by the average precision of the curves (seed 42) correlates with the published PR-AUC ranking at 0.90, below the 0.99 I had fixed, so the hypothesis is not supported: several detectors have medians within a hundredth of each other and swap places between one seed and five.

TABLE XI
MEDIAN OVER THE 100 LABELLED PROBLEMS OF THE F_1 AND OF THE SHARE OF FLAGGED ROWS (%), AT THE TRAINING-PREVALENCE THRESHOLD (PREV.) AND AT A CONSTANT CONTAMINATION OF 0.02 (FIXED).

Detector	F_1 prev.	F_1 fixed	Flag.% prev.	Flag.% fixed
Z-score	0.164	0.164	11.4	11.4
IQR	0.160	0.160	22.8	22.8
Great Expectations	0.052	0.052	1.6	1.6
Isolation Forest	0.189	0.068	10.0	2.1
LOF	0.163	0.052	10.1	2.0
One-Class SVM	0.214	0.072	10.6	2.4
ECOD	0.214	0.068	10.0	2.1
COPOD	0.199	0.073	10.0	2.2
HBOS	0.213	0.067	10.0	2.1
KNN	0.202	0.074	10.0	2.1
PCA	0.169	0.068	11.4	2.4
Autoencoder	0.190	0.100	12.4	3.0
VAE	0.184	0.069	10.0	2.2
Deep SVDD	0.193	0.101	14.0	4.3
DIF	0.217	0.120	13.6	2.9
AE1SVM	0.184	0.054	10.3	2.2
LUNAR	0.131	0.064	9.1	2.3

TABLE XII
ISOLATION FOREST ON THE 100 LABELLED PROBLEMS: MEDIAN PRECISION, RECALL, F_1 AND SHARE OF FLAGGED ROWS (%) OVER THE CONTAMINATION GRID.

Contamination	Prec.	Rec.	F_1	Flag.%
0.001	0.062	0.001	0.003	0.1
0.010	0.125	0.018	0.032	1.1
0.050	0.155	0.117	0.114	5.2
0.100	0.141	0.213	0.166	10.2
0.200	0.148	0.377	0.203	20.3

Table XII sweeps the contamination of Isolation Forest on the same problems. Recall rises with the contamination as on the three examples, and the median F_1 keeps rising up to the largest value of the grid, 0.2, twice the median prevalence of the pool (0.10). Precision stays between 0.14 and 0.16 from 0.05 on whatever the threshold, which points to weak scores on these labels more than to a badly placed threshold.

b) Typed injection and eligibility.: The eligibility rule of Section IV-I was applied to all 152 datasets, ADBench ones included (the normal class only for a labelled set): 58 are eligible, 42 from the UCI repository and 16 from ADBench. The others are listed with their reason in the archive; the usual one is fewer than four continuous columns (categorical or binary tables) or too few rows. Figure 4 and the stability figures of Section IV-I include the ADBench tables.

c) Severity.: Table XIII gives the median recall on multivariate outliers over the 58 eligible tables when the injected amplitude is multiplied by 0.5, 1, 2 and 4. The hypothesis that every detector gains recall between the smallest and the largest severity does not hold: 13 of 17 do, and Z-score, Great Expectations, LOF, HBOS do not. The Z-score, the Great Expectations bounds and HBOS stay at zero because they read one column at a time, and multivariate outliers stay inside the range of each column, as in the per-type matrices; LOF is already at its level at the smallest severity and does not move. LUNAR is the highest at every severity, which agrees with its behaviour on the per-type matrices.

d) The pipeline end to end.: The four stages (read, remove duplicates, detect, write) ran on all 152 datasets with Z-score and Isolation Forest, one fresh process per measurement; 14 of the 304 runs failed and are recorded with their reason (a table with no numeric column, or a column that the pipeline reads as text because of stray header lines in the file). On the 14 datasets above 50,000 rows the detection takes 62% of the pipeline time with Isolation Forest and 4% with the Z-score, the writing of the two files being the rest, as on Credit Card Fraud; the hypothesis holds. These times were measured while other jobs ran on the machine, so they are orders of magnitude, and the shares are ratios inside one process.

e) Chronological split.: A time variable is needed, and only some pool datasets have one: eleven unlabelled UCI

TABLE XIII
MEDIAN RECALL ON MULTIVARIATE OUTLIERS OVER THE ELIGIBLE TABLES, BY SEVERITY MULTIPLIER OF THE INJECTED AMPLITUDE.

Detector	×0.5	×1	×2	×4
Z-score	0.00	0.00	0.00	0.00
IQR	0.22	0.38	0.53	0.62
Great Expectations	0.00	0.00	0.00	0.00
Isolation Forest	0.24	0.33	0.36	0.40
LOF	0.33	0.30	0.32	0.32
One-Class SVM	0.16	0.17	0.18	0.20
ECOD	0.21	0.24	0.27	0.30
COPOD	0.20	0.22	0.25	0.28
HBOS	0.00	0.00	0.00	0.00
KNN	0.31	0.32	0.34	0.35
PCA	0.18	0.18	0.21	0.19
Autoencoder	0.25	0.27	0.30	0.29
VAE	0.18	0.19	0.21	0.19
Deep SVDD	0.22	0.23	0.23	0.23
DIF	0.25	0.35	0.36	0.44
AE1SVM	0.16	0.18	0.17	0.18
LUNAR	0.60	0.66	0.66	0.68

TABLE XIV
CHRONOLOGICAL SPLIT ON THE POOL DATASETS WITH A TIME VARIABLE: ANOMALIES IN THE TEST PARTITION, PREVALENCE IN TRAINING AND TEST (%), AND THE NUMBER OF DETECTORS WHOSE PR-AUC IS LOWER THAN UNDER THE STRATIFIED SPLIT.

Dataset	Test anom.	Train %	Test %	Lower
Credit Card	52	0.2	0.1	12 of 14
Bank Marketing	2668	5.8	39.3	0 of 16
Occupancy Detection	405	11.0	15.4	11 of 17
Online Shoppers	144	8.9	8.3	8 of 17
Room Occupancy	excluded: 0 test anomalies			

tables have a date, the ADBench files have none, and five labelled datasets qualify (Credit Card, Bank Marketing, Occupancy Detection, Online Shoppers and Room Occupancy, the rule being fixed beforehand). Table XIV gives the result of the chronological pilot with all the detectors that scale. The hypothesis that the chronological PR-AUC is below the stratified one for two thirds of the pairs does not hold: 31 of 64 pairs are lower. Bank Marketing is the reason: its prevalence rises from 5.8% to 39.3% and no detector loses PR-AUC there, as Section V-B found with four detectors. Room Occupancy is excluded because its anomalies are all in the oldest rows.

f) Files.: All 152 local files match the pinned manifest (`make verify-pool`), so the sixth hypothesis holds.

V. SECONDARY STUDIES

The four studies below are narrower than the per-type comparison of the previous section. Each uses one or two datasets or a synthetic stream, was written down before its run, and answers one question: whether a chronological split changes the ranking, whether a threshold chosen on the past survives a shift in prevalence, whether a sequence model adds anything, and what detection costs inside the pipeline. They are reported separately so that the main result does not rest on them.

TABLE XV
AVERAGE PRECISION (PR) ON THE TEST SPLIT, STRATIFIED (STRAT.) CONTROL VERSUS CHRONOLOGICAL (CHRON.) SPLIT WITH ITS 95% BLOCK-BOOTSTRAP INTERVAL, ROC-AUC OF THE CHRONOLOGICAL SPLIT, AND THE SHARE OF FUTURE ROWS FLAGGED AT THE TRAINING-PREVALENCE THRESHOLD.

Detector	PR, strat.	PR, chron.	ROC, chron.	Flag.%
Z-score	0.078 ± 0.015	0.048 [0.020, 0.087]	0.935	12.79
IQR	0.018 ± 0.003	0.013 [0.006, 0.023]	0.917	47.10
Great Expectations	0.183 ± 0.046	0.045 [0.013, 0.088]	0.758	3.45
Isolation Forest	0.110 ± 0.039	0.035 [0.016, 0.066]	0.936	0.08
ECOD	0.175 ± 0.035	0.075 [0.027, 0.146]	0.936	0.08
COPOD	0.212 ± 0.046	0.127 [0.045, 0.216]	0.936	0.07
HBOS	0.207 ± 0.038	0.151 [0.058, 0.268]	0.928	0.09
PCA	0.108 ± 0.117	0.002 [0.001, 0.003]	0.608	0.43

A. Chronological pilot on Credit Card

The results above use a random stratified split. Of the three example datasets, only Credit Card has a time variable (seconds since the first transaction, about two days); Cardio and Water Quality have none, so their row order says nothing about time. I therefore ran one small pilot on Credit Card, with the protocol written down before any score was computed on held-out data (`verification/temporal_pilot.md`). It is a separate experiment and does not replace the benchmark.

The rows are the same cleaned rows, and the detectors are the four of the pilot with default settings, plus ECOD, COPOD, HBOS and PCA, which the registry gained afterwards and which scale to this size (the rerun reproduces the first four rows exactly); the clock variable is removed from the features. The chronological split uses the usual $70/15/15$ ratios by time: the detectors are fitted on the first $198{,}609$ rows (366 frauds) and scored on the last $42{,}557$ rows (52 frauds, 0.12%). The control uses the stratified split of the main benchmark with the same rows and features, over five seeds. Intervals for the chronological arm come from a block bootstrap over 30-minute blocks of the test period.

Average precision is lower under the chronological split for all four detectors, but the interval excludes the stratified mean only for Great Expectations and Isolation Forest; for the Z-score and the IQR rule the difference is within the uncertainty of 52 test frauds. This exclusion criterion was fixed in advance as an exploratory check. It is not a confidence interval or a test for the difference between the two protocols: the bootstrap reflects only the chronological arm, the stratified mean is itself uncertain (five seeds), and the share of frauds changes between partitions (0.18% in training, 0.12% in the test period), so part of the gap may come from prevalence alone, which this pilot does not separate from the effect of time. The best detector also changes: Great Expectations leads the stratified control, while the Z-score has the highest chronological point estimate, with overlapping intervals. Isolation Forest keeps a high ROC-AUC (0.936) but flags only 0.08% of the future rows and none of the 52 frauds: the threshold set from training scores does not carry over to the later period. This is a single two-day dataset with few positives, so it indicates a direction and an order of magnitude, not a general rule.

The four detectors added later behave differently from one another. ECOD and COPOD lose part of their stratified average precision (0.175 to 0.075 and 0.212 to 0.127) and, like Isolation Forest, flag under 0.1% of the future rows; they find one fraud and three of the 52 (recall 0.02 and 0.06). HBOS keeps the highest chronological average precision of the table (0.151, interval $[0.058, 0.268]$ overlapping the others'), with a precision of 0.26 and a recall of 0.19 on 0.09% of the rows: the first detector of this study whose training-prevalence threshold still catches frauds in the later period. PCA collapses (0.002, ROC-AUC 0.61), with no fraud found. The intervals are wide, so this shows that the column-wise detectors are not all alike under a time split, not that HBOS is better.

To check the window monitor with a known change, a mean shift of δ training standard deviations was added to five features from the middle of the test period. The alert threshold was fixed from the validation period before the test (the largest validation window score, 0.91). Shifts of 1 and 2 standard deviations were detected in the second window after the change (573 seconds), and a shift of 0.5 was not detected; no window before the change raised an alert (22 windows). Credit Card has no labelled change in distribution, so I make no claim about its natural drift.

B. Decisions under temporal shift

The pilot above leaves two questions open: whether a threshold chosen on the past can be repaired without labels, and how much of the gap is only a change in prevalence. I ran a second study on Credit Card and on Bank Marketing, a UCI table (CC BY 4.0) whose file is ordered by date from May 2008 to November 2010, with the protocol and four hypotheses written beforehand (`verification/temporal_decision.md`). The Credit Card results were already known, so this study is an independent test only for Bank Marketing. The Kyoto-2006+ traffic used by AnoShift was left out, because I could not verify its data rights nor download it in time. The date of Bank Marketing is a month, with the file order inside each month. Four detectors (Z-score, IQR, Great Expectations, Isolation Forest) are fitted on the oldest 70% of the rows and scored on the newest 15%. A threshold is a quantile of the scores that keeps the training prevalence, taken (P0) from the training scores, (P1) from the validation scores, or (P2) from the scores of the two previous blocks of the test period, without any label of that period. A stratified control, over five seeds, is also matched to the prevalence of the chronological test.

On Bank Marketing the prevalence moves a lot: 5.8% in training, 11.5% in validation and 39.3% in the test period (Credit Card: 0.18%, 0.13% and 0.12%). The average precision under the chronological split is higher than under the stratified one for every detector (0.55 against 0.17 for the Z-score, 0.42 against 0.19 for Isolation Forest), but the ROC-AUC of the chronological split is between 0.48 and 0.63, close to chance. Matching the control to the test prevalence brings its average precision to 0.40–0.52, so the gap, a median of 0.26, falls to 0.06: on this dataset the apparent gain is a change in prevalence, not in detection. On Credit Card the same matching reduces the median gap from 0.053 to 0.031, which is less than half of it, so a smaller part of the gap is explained by prevalence there.

The ranking of the detectors is not stable. The Spearman correlation between the two orderings by average precision is 0.80 on Bank Marketing and 0.40 on Credit Card, below the 0.8 expected on both. The three thresholds do not behave as I expected. On Bank Marketing the share of rows flagged moves closer to the training prevalence from P0 to P1 to P2 (median error $|\ln(\text{share}/\pi)|$ of 1.51, 1.02 and 0.66), but on Credit Card P1 is no better than P0 (0.78 against 0.73), so the ordering holds on one dataset only. The reason is clearer in the decisions. On Bank Marketing, P0 flags 41% of the test rows with the Z-score and 37% with Isolation Forest, because the later rows score higher than the training rows, which gives an F_1 of 0.51 and 0.41 by accident of a prevalence of 39%; the rolling threshold targets a share of 5.8%, flags 1.7% and 6.3% of the rows and drops the F_1 to 0.02 and 0.11. A label-free recalibration that keeps the training prevalence cannot follow a prevalence that changes. On Credit Card the opposite holds: no detector detects any of the 52 frauds with P0 ($F_1 = 0$), while the rolling threshold gives a recall of 0.12 for Great Expectations and 0.14 for the Z-score, at a precision below 0.08. The F_1 of P2 is at least that of P0 for 5 of the 8 pairs, which satisfies the hypothesis, but four of the five are Credit Card pairs where P0 is zero. Overall one hypothesis of four holds. This study does not establish a rule for thresholds under shift: it shows that a threshold fixed on the past can fail in both directions, that recalibrating without labels needs an estimate of the prevalence, and that part of what a chronological split changes is only the prevalence.

C. LSTM autoencoder pilot

A last pilot asks whether the preceding rows of the transaction stream carry a fraud signal that a row-level autoencoder cannot see. Credit Card has no cardholder identifier, so the window is the global stream of the ten previous rows, not a customer's history. The protocol was fixed beforehand (`verification/lstm_pilot.md`). The comparison uses the chronological partitions of Section V-A, the same $50{,}000$ training rows and a CPU budget of at most 20 epochs for both models; the architecture (window of 10 rows, 32 units) was not tuned, and the validation rows served only for early stopping on an unlabelled reconstruction loss.

As a check before reading the real data, a synthetic stream whose anomalies break the continuity of an autoregressive signal was scored by both models: the LSTM reaches an average precision of 0.875 against 0.006 for the row-level autoencoder. On Credit Card, over five initialisation seeds, the LSTM reaches an average precision of 0.037 ± 0.006 on the 52 test frauds against 0.024 ± 0.010 for the row-level autoencoder (ROC-AUC 0.926 against 0.841). The paired block-bootstrap interval of the difference excludes zero for the main seed ($+0.021$, $[+0.005, +0.044]$) but not for the other four, where the difference is positive for three seeds and negative for one.

TABLE XVI
MEDIAN DETECTION-STAGE TIME ON SYNTHETIC DATA (8 FEATURES) FOR 10^4, 10^5 AND $3 \cdot 10^5$ ROWS, THE EMPIRICAL GROWTH EXPONENT OVER THE SIZES COMPLETED, AND THE PEAK RESIDENT MEMORY ADDED BY DETECTION AT THE LARGEST SIZE (A: AT $3 \cdot 10^4$ ROWS, THE LARGEST SIZE COMPLETED).

Detector	10^4	10^5	$3 \cdot 10^5$	Exp.	Mem. (MB)
Z-score	0.002 s	0.013 s	0.039 s	0.64	70
IQR	0.004 s	0.029 s	0.086 s	0.70	116
Isolation Forest	0.16 s	1.09 s	3.2 s	0.72	140
Autoencoder (MLP)	1.5 s	8.4 s	12.7 s	0.77	115
LSTM autoencoder	1.1 s	5.1 s	7.3 s	0.73	499
LOF	0.73 s	18.0 s	133 s	1.51	405
One-Class SVM	5.6 s	$>$ 300 s	not run	1.99	788[a]

This criterion was set in advance as an exploratory check, not a formal test. The LSTM is comparable to Isolation Forest (0.035) and Great Expectations (0.045) on the same rows, not clearly better. None of the five trainings converged within the budget: the validation loss was still decreasing at the 20th epoch. As for the other detectors, the threshold taken from training scores flags almost none of the frauds in the later period (recall at most 0.02). The result is therefore an indication from a small, budget-limited experiment on one dataset, not a ranking.

D. Cost of the detection stage and of the full pipeline

The growth experiment reported later in this section uses only 100 to 900 rows and cannot be extrapolated. I therefore measured the detectors again on a protocol fixed in advance (`verification/cost_study.md`): synthetic Gaussian matrices of 1,000 to 300,000 rows and 8 features, five repetitions up to 30,000 rows and three above, a budget of 300 seconds per measurement, and one fresh process per measurement so that the peak resident memory, native allocations included, belongs to that run alone. The detector is fitted and applied to the same rows, so this is a cost measurement, not a quality one. Times are those of the detection stage (fit, predict and score).

The rule-based detectors and Isolation Forest grow roughly linearly (exponents below one at small sizes reflect fixed start-up costs), LOF grows with an exponent near 1.5 and finished $3 \cdot 10^5$ rows in about 133 seconds, and One-Class SVM, with an exponent near two, exceeded the budget at 10^5 rows and was not tried beyond. The LSTM autoencoder was trained for at most five epochs on at most 50,000 rows, a budget cap: only its scoring cost grows with the input after that. These exponents describe the sizes measured and are not extended beyond them. No measurement was made under contention as defined in the protocol, but the machine was not isolated.

To separate the detection stage from the whole pipeline, I also timed read, cleaning, detection and write on the full Credit Card file (284,807 rows read, 283,726 after removing duplicates). With the Z-score the pipeline takes about 4.8 seconds, of which detection is 0.13 seconds (3%) and writing the two sorted files is 3.9 seconds; with Isolation Forest it takes about 8.5 seconds, of which detection is 3.7 seconds (43 to 45%). Peak memory is about 0.75 GB and 0.95 GB respectively. Detection-stage latency is therefore not the latency of the whole pipeline, and the gap depends on the detector. These are single-machine figures on synthetic data and one real file, not a general ranking.

VI. DISCUSSION

a) Rules and learned models are complementary, not competing.: The per-type heatmap shows limited overlap between what the declarative rules catch (missing values), what the statistical rules catch (single-column outliers), and what the learned models add (implausible combinations, recall 0.89 for Isolation Forest against ≤ 0.04 for the rules, and 1.00 for DIF and 0.92 for the VAE among the neural models). Broad coverage needs more than one family, not the single best one on aggregate. This is a different conclusion from benchmarks that publish a single ranking.

b) The operating point matters as much as the algorithm.: Setting Isolation Forest's contamination to the data prevalence rather than a fixed 0.10 moves its fraud precision from 0.01 to 0.28 (the seed of Table VI; 0.20 on average over five seeds) and its multivariate-outlier recall on the synthetic data from about 0.3 to 0.9 (means over five seeds). The same estimator looks weak or strong depending on one number that a benchmark must choose deliberately.

c) Unsupervised detection works in one of the three regimes.: The three example datasets bracket the method: on Cardio, where anomalies are rare and distributionally distinct, Isolation Forest reaches $F_1 = 0.62$ and PR-AUC 0.70; on Credit Card Fraud, where the positive class is a 0.17% mixed-defect label, the best F_1 of the seventeen detectors is 0.35 (Deep SVDD, precision 0.26) and HBOS reaches the best precision (0.34) at a recall of 0.32; on Water Quality, where the "anomaly" is a 61% majority, every method is at chance. The question is less "rules or ML" than "does this dataset have rare, outlying anomalies at all".

d) The profile transfers in part, and the decision does not.: The later studies test whether the synthetic profile is an artefact of one generator. Two further schemas keep the ordering of the methods (rank correlation 0.93), so the competence of each family is not specific to the customer table; but the multivariate and drift cells depend on how the columns are scaled, so their values do not transfer. On the 42 real tables the structural rules still own their types while the multivariate gap shrinks, and on the 100 labelled problems the medians of PR-AUC of One-Class SVM (0.240), Isolation Forest (0.215) and the Z-score (0.203) overlap, so the advice "use a learned model" is not supported there. The neural models add little on aggregate: the best of them (DIF, 0.226) stays below the best classical median (0.240), and the LSTM autoencoder recalls 0.25 of the drifted rows against 0.50 for LOF; LUNAR is the exception on drift (0.65 on the customers), at the price of flagging 41% of the clean rows. Last, a ranking that holds under a chronological split does not give a usable threshold: on Bank Marketing the prevalence goes from 5.8% to 39.3%, matching the control to it removes most of the apparent gain

(0.26 to 0.06), and a label-free rolling threshold cannot follow it. A practitioner should therefore choose the family from the type of fault and monitor the prevalence before trusting any alert rate.

e) A larger registry does not produce a winner.: The registry now holds three rules, eight classical models and six neural ones. On the 100 labelled problems every one of the seventeen is first on at least two, and no method is first on more than 13. The column-wise detectors ECOD, COPOD and HBOS cost little, match Isolation Forest on the pool (0.240, 0.216 and 0.237 against 0.215) and beat it on Credit Card, but they cannot see an implausible combination of ordinary values; on the synthetic data they only do because the injected combinations also push one value to a tail. The neural models help on specific types (DIF and the VAE on multivariate outliers, LUNAR on drift) and not on aggregate scores, and they cost more to fit and to tune. For a practitioner this suggests a cheap column-wise model and Isolation Forest as the baseline, a reconstruction model added when the faults are combinations, and a model such as LUNAR only when missing drift costs more than false alarms. These are default settings, and a tuned neural model could do better.

f) Aggregate metrics on a mixed label are misleading.: On every dataset the aggregate table compresses different behaviour into similar F_1 and PR-AUC values. A method excellent on one anomaly type and blind to the rest scores like a mediocre all-rounder. The controlled per-type dataset, and the severity sweep that tests the sensitivity of that ranking, are what make the difference measurable.

g) Pipeline integration changes the shortlist.: Latency and scaling matter, but the timing in these runs is too variable to establish a reliable speed ranking. The One-Class SVM and LOF were omitted from the full fraud dataset because their computational costs need a separate scaling study. I time only the detection stage (fit, predict and score) inside the pipeline; extraction, transformation and loading are outside the measurement, so these figures are not the latency of a whole ETL run.

h) Threats to validity of the results.: The synthetic dataset still encodes my own assumptions about what each anomaly type looks like; the severity sweep addresses the magnitude choice but not the shape of the injections. The real-data pool answers part of this, but its anomalies are either injected (the 42 tables) or taken from a rare class (the 100 labelled problems), so none of them is a data-quality fault observed in a production pipeline. The ML detectors carry a wide seed spread on the small datasets (Cardio F_1 standard deviation up to 0.17; Isolation Forest on fraud $F_1 = 0.20 \pm 0.07$), so those numbers should be read as ranges. The labels of the 64 UCI problems come from a rule (the rarest class), 19 sources appear in both lists of the pool, and the datasets are therefore not independent.

i) Limits of the verification.: Verification has its own limits: complete line and branch coverage does not establish that the scientific assertions detect every defect. The previous archive was made on a working tree with uncommitted changes (revision `841c3aa`). The archived runs were therefore replayed from a clean commit (`93b6fe8`): every detection metric, curve and partition is identical to the previous archive and only the timings changed, which is why the two timing tables and the scaling figures above were updated. The new records name that commit and mark the working tree as clean, but they were produced on one machine and the timings do not transfer to another. Mutation testing reaches the target on the pipeline modules and the scripts, but it cannot show whether the injected defects and the tests cover every way an experiment can go wrong. The complete real-data checks were performed with the three files available locally. The Water Quality file matches its named mirror, and the Cardio CSV can be reproduced from the current ADBench archive. The Credit Card file has the size of the current Kaggle version (3, March 2018), but I could not compare its content hash with a fresh download. These gaps leave some implementation errors capable of changing a published result without being caught by the current gates.

VII. Conclusion

I built an ETL pipeline in which three rule-based detectors, eight classical machine-learning detectors and six neural ones share one interface and are scored on the same held-out path, over five seeds, with the operating point read from the data. A controlled dataset labels six anomaly types separately, and a severity sweep shows where the per-type ranking changes as the injections are scaled. On aggregate metrics the detector families stay close on the example and synthetic datasets, and on 100 labelled problems of a pool of 152 open tables no detector is clearly ahead (Section IV-J). The per-type view is where they diverge: declarative rules own missing values, statistical rules own single-column outliers, tree-, boundary- and reconstruction-based models (Isolation Forest, the One-Class SVM, DIF, the VAE) catch implausible combinations, and duplicates and temporal drift need dedicated stages, LUNAR being the one learned model that helps on drift, at the price of many false alarms. Across the three example datasets, unsupervised detection pays off only on Cardio, where the anomalies are genuinely rare and outlying. The choice of contamination, the precision-recall trade-off it selects, and the pipeline cost further constrain which detector fits a real flow.

On real tables the pattern is less clean than on the generator. With the six types injected into 42 tables, Great Expectations and the deduplication rule still own their types, but the multivariate gap shrinks and the z-score collapses for a reason that has to do with its training data, not its design. On 100 labelled problems no detector is clearly ahead among the seventeen, learning helps only by a small margin, and the neural models do not beat the best classical one. Two further schemas keep the profile (rank correlation 0.93) but change how visible multivariate outliers and drift are, and an LSTM autoencoder trained to convergence does not catch temporal drift better than the row-level detectors. The chronological pilot and the study of decisions under temporal shift, on Credit Card and Bank Marketing, point the same way: score rankings

survive a split by time better than a threshold fixed on the past. On Bank Marketing, where the prevalence goes from 5.8% to 39.3%, matching the control to it cuts the apparent gain of the chronological split from 0.26 to 0.06 of PR-AUC, and a label-free rolling threshold that keeps the training prevalence cannot follow it; only one of the four hypotheses of that study holds.

The next steps are a check of the injected faults against faults observed in real pipelines, and a decision-level study on more datasets with genuine time, such as the Kyoto-2006+ traffic, whose data rights I have not verified. The open questions are listed in Section VI.

Appendix A
Verification design

The experimental code is checked at three levels. A routine test suite exercises numerical contracts on small, generated tables and measures line and branch coverage across the pipeline and experiment scripts. Its regression cases include partition disjointness by customer identity, training-only preprocessing, the exclusion of labels from detector inputs, score direction and index alignment, undefined binary metrics, and the construction of each synthetic anomaly type. Separate command-line tests run the four figure and table generators on temporary data. They check the output schema without comparing images pixel by pixel or imposing a fixed wall-clock duration.

A second, manually invoked level asks whether these checks can fail. Mutation testing generates small source-level changes (a reversed comparison, an altered constant, a modified message) and counts how many make the routine suite fail. I apply it to the modules that compute a result and to the experiment scripts. In addition, twenty-three deliberate scientific defects are applied one at a time to a temporary copy of the code, and a named test must fail on each: a scaler refitted on the rows being scored, the label passed to the detector as a feature, a contamination threshold read from the test labels, a training row copied into the test partition, an inverted score orientation, ranking metrics computed from the binary flags, a seed aggregate that keeps only the first seed, a detector missing from the results table, and a few others.

A third level uses the locally available full datasets. A separate command compares, for each of the three research CSV files, the file checksum, the raw and post-deduplication row counts, the number of anomalies and the positive-class value with an archived manifest. An absent file is reported as not run, and the command as a whole passes only if all three files pass. The scripts record the dataset fingerprint, software versions, random seeds, split policy and output names alongside their results. For each real-data seed, an archive stores the positions of the train, validation and test rows after deduplication; a capped fit also records the selected training positions. Archived tables and figures are checked against their manifest, and the six numerical tables in this manuscript are compared with the archived CSV values at the precision shown. This establishes traceability between the reported values and the recorded experiment; it does not establish that another machine will reproduce the same timings.

A fourth part turns these checks into a release verdict. Each profile (routine suite, mutation campaign, fault injection, real-data check, coverage of the checking scripts themselves, static typing, and a controlled reproduction) writes a small evidence record tied to the source revision, to a clean working tree, to a fingerprint of the code that computes results (the pipeline and the experiment scripts, not the checking scripts) and to the software versions. A release script recomputes every status from the raw results instead of reading it from the record, requires that all profiles describe the same revision and compatible environments, and returns *certified*, *incomplete* or *rejected*; figures quoted in this manuscript are accepted only if the run they come from is not older than a change to the code. The real-data check can also run on a hosted Linux machine: it downloads the three public files, compares each SHA-256 value with the archived manifest before using it, and refuses any other file. The controlled reproduction replays the cardiotocography benchmark against the archive, twice in the same environment and once in an environment rebuilt from a recorded, hash-pinned lock; on another operating system the metrics agree statistically (largest difference 6.5×10^{-5}, identical partitions) rather than bit for bit. Result files are written to a temporary name and renamed only when complete, so an interrupted run cannot leave a truncated table or figure. The verdict states where the evidence was produced and never that it was independently attested: the author wrote the code and launched every run.

I treat a failed or missing check as a limitation of the evidence, not as a successful experiment. Complete line and branch coverage and a mutation score of at least 95% over the pipeline and the scripts are the targets, and the measured values, with what they leave out, are reported in Appendix B. The chronological split and the window-based drift monitor have a separate protocol: their alert and delay measures are not pooled with the row-level metrics from the stratified benchmark.

Appendix B
Verification outcomes

At commit e1fcbb0d (local run, not independently attested) the routine suite runs 1,150 tests, plus three that need the full datasets and twenty-three fault-injection runs, all run separately. Coverage of the pipeline modules and the experiment scripts is complete: 1,628 of 1,628 executable lines and 336 of 336 branches. The tools that check the checks (the evidence and release scripts under `verification/`) are measured apart, at 1,975 of 1,975 lines and 724 of 724 branches, and are not part of that figure. The mutation campaign on the pipeline modules and the experiment scripts kills 4,700 of 4,808 mutants (97.75%). Split by what the code does, the modules that compute a result (splitting, metrics, the synthetic generator, the detectors) kill 1,647 of 1,720 (95.76%) and the input, output and script code kills 3,053 of 3,088 (98.87%), so the aggregate is higher than the score of the scientific

computation alone. The 108 survivors count as failures. I classified them by reading the changes, not by proof: most pass an explicit default (`dtype`, `index`) or change an internal label. Two that I first took for unreachable, a cap on a simulated transaction rate, turned out to be reachable by a very rich customer and are now killed by a test.

All twenty-three deliberate defects are caught. The first fourteen were not always caught by the tests I started with: four went through every existing test (a scaler refitted on the scored batch, ranking metrics fed with binary flags, a detector missing from the results, and a contamination threshold read from the synthetic test partition), and a fifth, an inverted Z-score, was caught only by a test added during the mutation work. None of these defects was in the code; the tests were missing. After adding them I reran every experiment: all detection metrics are unchanged and only the timings differ. The nine defects added later (a chronological split that trains on the latest events, validation rows copied into training, an undefined precision reported as one, inverted machine-learning scores, scores reordered before the comparison with the labels, a contamination read from train and test labels, a silently dropped seed, a real-data check that passes without checking the files, and an interrupted mutation campaign accepted on its score) were each caught by a named test. The real-data checks pass locally on the three CSV files, and the six tables of this manuscript match the archived results.

The same profiles were run on a hosted Linux machine for the final revision (commit c64c6ffa, a continuous-integration run, not independently attested). The release script returned *certified*. The routine suite and the scripts keep complete line and branch coverage, the checking scripts are fully covered ($1{,}975$ of $1{,}975$ lines), and the mutation campaign, run from scratch, kills $4{,}703$ of $4{,}808$ mutants (97.82%) with no timeout. Earlier hosted runs reported fourteen timeouts, always the same mutants. They were not hangs in the mutated code: the mutation tool starts worker processes after the numerical libraries have created their thread pools, which blocks on Linux. Running the tool with single-threaded libraries removed them, and the same four mutants then gave the same outcomes as on macOS. The three real datasets were downloaded and matched the archived checksums, and the controlled reproduction passed at its three levels. These results show that the evidence is consistent and re-creatable on a second platform; they do not certify the scientific conclusions of the study, which rest on the limits stated with each experiment.

Detector timing in Tables II and IV sums `fit`, `predict` and `score` inside the detection stage; CSV loading, deduplication and figure rendering are excluded. A separate local growth experiment ran Isolation Forest, LOF and One-Class SVM on 100, 300 and 900 generated rows, with eight features and three repetitions per size. All 27 runs completed. At 900 rows, median measured detection times were 0.392, 0.020 and 0.049 seconds, respectively, on the recorded macOS/ARM environment. This small in-memory experiment measures neither native-library peak memory nor the scaling of the 284,000-row fraud dataset. It supports reporting the workload and omissions explicitly, but not a general speed ranking.

TABLE XVII
SYNTHETIC CUSTOMER TABLE, MEAN RECALL OVER FIVE SEEDS.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	0.93	0.00	0.00	0.24
IQR	0.05	0.04	0.99	0.04	0.25	**1.00**
Great Expectations	**1.00**	0.00	0.21	0.00	0.00	0.00
Isolation Forest	0.09	0.11	0.99	0.89	0.33	0.60
LOF	0.28	0.16	0.70	0.21	0.50	0.67
One-Class SVM	0.07	0.10	**1.00**	0.90	0.29	0.54
ECOD	0.10	0.13	0.68	0.52	0.26	0.36
COPOD	0.09	0.13	0.89	0.51	0.34	0.30
HBOS	0.00	0.00	0.90	0.00	0.00	0.32
KNN	0.09	0.10	**1.00**	0.39	0.41	0.81
PCA	0.10	0.12	**1.00**	0.48	0.23	0.58
Autoencoder	0.12	0.10	0.81	0.83	0.29	0.53
VAE	0.07	0.11	0.99	0.92	0.30	0.66
Deep SVDD	0.19	0.18	0.33	0.13	0.16	0.32
DIF	0.09	0.07	**1.00**	**1.00**	0.33	**1.00**
AE1SVM	0.09	0.10	0.99	0.78	0.27	0.61
LUNAR	0.44	0.39	**1.00**	0.63	**0.65**	0.90
LSTM autoencoder	0.08	0.09	0.97	0.68	0.25	0.53
Drift detector	0.00	0.00	0.00	0.00	0.00	0.00
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

TABLE XVIII
SYNTHETIC CARD TRANSACTIONS, MEAN RECALL OVER FIVE SEEDS.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	0.63	0.00	0.00	0.26
IQR	0.09	0.14	0.98	0.57	0.37	0.63
Great Expectations	**1.00**	0.00	0.14	0.00	0.00	0.00
Isolation Forest	0.09	0.18	0.97	0.49	0.38	0.56
LOF	0.11	0.15	0.72	0.64	0.30	0.45
One-Class SVM	0.10	0.26	0.89	0.18	0.36	0.39
ECOD	0.10	0.20	0.67	0.45	0.29	0.67
COPOD	0.09	0.20	0.86	0.43	0.33	0.45
HBOS	0.00	0.00	0.69	0.00	0.00	0.26
KNN	0.11	0.17	0.97	0.33	0.39	0.57
PCA	0.08	0.19	0.86	0.16	0.27	0.37
Autoencoder	0.09	0.19	0.84	0.17	0.30	0.46
VAE	0.07	0.24	0.86	0.21	0.34	0.39
Deep SVDD	0.12	0.14	0.74	0.70	0.27	0.54
DIF	0.09	0.20	0.97	0.37	0.36	0.60
AE1SVM	0.09	0.24	0.88	0.16	0.34	0.40
LUNAR	0.22	0.29	**0.99**	**0.73**	**0.55**	**0.75**
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

APPENDIX C
FULL PER-TYPE MATRICES

The tables below give the mean recall of every detector of the registry on each injected type, over five seeds for the synthetic tables and as the median over the 42 real tables for the last one. The best value of each column among the detectors of the registry is in bold.

REFERENCES

[1] V. Khatri and C. V. Brown, "Designing data governance," *Communications of the ACM*, vol. 53, no. 1, pp. 148–152, 2010.

TABLE XIX
SYNTHETIC SENSOR READINGS, MEAN RECALL OVER FIVE SEEDS.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	**1.00**	0.00	0.00	0.26
IQR	0.03	0.05	**1.00**	0.02	0.94	**1.00**
Great Expectations	**1.00**	0.00	0.23	0.00	0.00	0.00
Isolation Forest	0.08	0.12	**1.00**	0.32	0.96	0.97
LOF	0.19	0.18	0.33	0.34	0.40	0.27
One-Class SVM	0.08	0.10	**1.00**	0.33	0.94	**1.00**
ECOD	0.10	0.13	0.81	0.40	0.80	0.58
COPOD	0.07	0.13	0.97	0.45	0.96	0.46
HBOS	0.00	0.00	**1.00**	0.00	0.00	0.26
KNN	0.11	0.10	**1.00**	0.26	**1.00**	**1.00**
PCA	0.09	0.12	**1.00**	0.26	0.63	0.95
Autoencoder	0.07	0.11	0.93	0.62	0.71	0.94
VAE	0.07	0.09	**1.00**	0.25	0.94	**1.00**
Deep SVDD	0.14	0.14	0.82	0.19	0.25	0.59
DIF	0.05	0.06	**1.00**	**0.99**	0.80	**1.00**
AE1SVM	0.08	0.10	**1.00**	0.49	0.80	**1.00**
LUNAR	0.32	0.30	**1.00**	0.41	**1.00**	**1.00**
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

TABLE XX
TYPED INJECTION INTO 42 REAL TABLES, MEDIAN RECALL OVER THE TABLES.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	0.00	0.00	0.00	0.43
IQR	0.19	0.21	**1.00**	0.39	0.64	**0.93**
Great Expectations	**1.00**	0.00	0.23	0.00	0.00	0.42
Isolation Forest	0.11	0.13	0.61	0.37	0.45	0.59
LOF	0.28	0.13	0.68	0.33	0.34	0.70
One-Class SVM	0.10	0.13	0.75	0.17	0.31	0.73
ECOD	0.10	0.14	0.44	0.25	0.36	0.44
COPOD	0.10	0.15	0.45	0.23	0.48	0.41
HBOS	0.00	0.01	0.04	0.02	0.01	0.32
KNN	0.19	0.10	0.91	0.30	0.46	0.80
PCA	0.12	0.14	0.54	0.19	0.23	0.71
Autoencoder	0.11	0.12	0.76	0.29	0.36	0.76
VAE	0.10	0.13	0.72	0.18	0.27	0.72
Deep SVDD	0.14	0.13	0.72	0.23	0.37	0.70
DIF	0.11	0.12	0.81	0.28	0.39	0.78
AE1SVM	0.11	0.14	0.63	0.17	0.18	0.76
LUNAR	0.38	0.25	0.98	**0.63**	**0.78**	0.90
Drift detector	0.00	0.00	0.00	0.00	0.00	0.00
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

[2] Y. Zhou, F. Tu, K. Sha, J. Ding, and H. Chen, "A survey on data quality dimensions and tools for machine learning," *arXiv preprint arXiv:2406.19614*, 2024. [Online]. Available: https://arxiv.org/abs/2406.19614

[3] R. Gopa, "AI-augmented ETL pipelines for automated data quality, anomaly detection and governance," *International Journal of Computational and Experimental Science and Engineering (IJCESEN)*, vol. 11, no. 4, pp. 7920–7927, 2025.

[4] D. Marupaka and S. Rangineni, "Machine learning-driven predictive data quality assessment in ETL frameworks," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 72, no. 3, pp. 53–60, 2024.

[5] M. R. Sokkula and S. K. Vuppala, "Implementing machine learning for ETL data transformation and anomaly detection," *International Journal for Multidisciplinary Research (IJFMR)*, vol. 6, no. 6, 2024.

[6] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://openreview.net/pdf?id=foA_SFQ9zo0

[7] Y. Zhao, Z. Nasrullah, and Z. Li, "PyOD: A python toolbox for scalable outlier detection," *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 2019. [Online]. Available: https://jmlr.org/papers/v20/19-011.html

[8] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLoS ONE*, vol. 11, no. 4, p. e0152173, 2016.

[9] Great Expectations, "Great expectations," Open-source data validation library. [Online]. Available: https://greatexpectations.io

[10] A. Mbata, Y. Sripada, and M. Zhong, "A survey of pipeline tools for data engineering," *arXiv preprint arXiv:2406.08335*, 2024. [Online]. Available: https://arxiv.org/abs/2406.08335

[11] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422.

[13] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 93–104, 2000.

[14] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[15] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[16] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," 2016, iCML 2016 Anomaly Detection Workshop.

[17] E. F. Agyemang, "Anomaly detection using unsupervised machine learning algorithms: A simulation study," *Scientific African*, vol. 26, p. e02386, 2024.

[18] M. Dragoi, E. Burceanu, E. Haller, A. Manolache, and F. Brad, "AnoShift: A distribution shift benchmark for unsupervised anomaly detection," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[19] A. Anwar, B. Moser, D. Herurkar, F. Raue, V. Hegiste, T. Legler, and A. Dengel, "FedAD-Bench: A unified benchmark for federated unsupervised anomaly detection in tabular data," 2024.

[20] S. H. Sun, A. Sankararaman, and B. M. Narayanaswamy, "Online adaptive anomaly thresholding with confidence sequences," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 47 105–47 132. [Online]. Available: https://proceedings.mlr.press/v235/sun24h.html

[21] Z. Li, Y. Zhao, X. Hu, N. Botta, C. Ionescu, and G. H. Chen, "ECOD: Unsupervised outlier detection using empirical cumulative distribution functions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 181–12 193, 2023.

[22] Z. Li, Y. Zhao, N. Botta, C. Ionescu, and X. Hu, "COPOD: Copula-based outlier detection," in *2020 IEEE International Conference on Data Mining (ICDM)*, 2020, pp. 1118–1123.

[23] M. Goldstein and A. Dengel, "Histogram-based outlier score (HBOS): A fast unsupervised anomaly detection algorithm," in *KI-2012: Poster and Demo Track*, 2012, pp. 59–63.

[24] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 427–438.

[25] M.-L. Shyu, S.-C. Chen, K. Sarinnapakorn, and L. Chang, "Principal component-based anomaly detection scheme," in *Foundations and Novel Approaches in Data Mining*, ser. Studies in Computational Intelligence. Springer, 2006, pp. 311–329.

[26] S. Chen, Z. Qian, W. Siu, X. Hu, J. Li, S. Li, Y. Qin, T. Yang, Z. Xiao, W. Ye, Y. Zhang, Y. Dong, and Y. Zhao, "PyOD 2: A Python library for outlier detection with LLM-powered model selection," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 2807–2810.

[27] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *2nd International Conference on Learning Representations (ICLR)*, 2014, arXiv:1312.6114.

[28] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, "Deep one-class classification," in *Proceedings of the 35th International Conference on Machine Learning*,

ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4393–4402.

[29] H. Xu, G. Pang, Y. Wang, and Y. Wang, "Deep isolation forest for anomaly detection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 591–12 604, 2023.

[30] M.-N. Nguyen and N. A. Vien, "Scalable and interpretable one-class SVMs with deep learning and random Fourier features," in *Machine Learning and Knowledge Discovery in Databases*, ser. Lecture Notes in Computer Science. Springer, 2019, pp. 157–172.

[31] A. Goodge, B. Hooi, S.-K. Ng, and W. S. Ng, "LUNAR: Unifying local outlier detection methods via graph neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 6, 2022, pp. 6737–6745.

[32] Machine Learning Group – ULB, "Credit card fraud detection," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[33] D. Campos and J. Bernardes, "Cardiotocography," UCI Machine Learning Repository, 2000, names as listed by the repository.

[34] A. Kadiwal, "Water quality," Kaggle dataset, 2021. [Online]. Available: https://www.kaggle.com/datasets/adityakadiwal/water-potability